



**Universidad
Europea** MADRID

PROYECTO INTEGRADOR

1DAW

Análisis y desarrollo de software gestor de
operaciones CRUD



Breixo García Canovacas

Robinson Tamayo Guerrero

Romeo Rey Alonso

Sara Candeña Carpio

Índice

Resumen.....	2
1. Introducción	3
2. Objetivos	4
2.1. Generales.....	4
2.2. Específicos	4
3. Tecnologías utilizadas.....	6
4. Desarrollo e implementación.....	7
5. Metodología	45
6. Resultados y conclusiones.....	48
7. Trabajos futuros	50
Versión cliente de la aplicación.....	50
Sistema de calificación por estrellas y ranking de empleados.....	50
Calendario interactivo de citas.....	51
Anexos	52
Anexo I – Listado de requisitos de la aplicación	52
Anexo II – Guía de uso de la aplicación	53
Anexo III – Desarrollo corporativo	54
Anexo III – Librería externa para gestión de fechas y horas	56
III.1. Criterios de selección	56
III.2. Configuración del DatePicker	56
III.3. Configuración del TimePicker.....	57
III.4. Integración con la capa Modelo.....	58
III.5. Importación de la librería.....	59
III.6. Beneficios obtenidos	59

Resumen

Este documento recoge el proceso de desarrollo de una aplicación de gestión para el taller del personaje ficticio Edna Moda de la saga de películas *los increíbles*. La herramienta permite administrar citas, clientes, trajes, empleados y talleres, cubriendo todas las operaciones de creación, lectura, modificación y eliminación. Esta aplicación está construida en el lenguaje de programación Java con interfaz gráfica Swing, se apoya en una base de datos MySQL y sigue el patrón Modelo-Vista-Controlador con integración de elementos externos como la librería LGoodDatePicker, que aporta componentes visuales intuitivos y compatibles con el formato requerido por la base de datos. El equipo ha trabajado bajo la metodología ágil SCRUM, organizando el trabajo en cuatro sprints de dos semanas cada uno, que significó una evolución controlada y una integración continua de funcionalidades. El resultado es un software funcional y estable que diferencia los permisos según la categoría del empleado (aprendiz, oficial o maestro), ofrece validaciones frente a solapamientos de citas y conflictos entre héroes y villanos, y proporciona una experiencia de usuario clara y profesional. El presente documento describe desde los requisitos iniciales y el modelado de datos hasta la implementación de las capas del MVC, pasando por las pruebas, la metodología y los anexos técnicos.

Palabras clave: Gestión; CRUD; MVC; SCRUM; Java; MySQL; Interfaz; Planificación; Usuario; Programación.

1. Introducción

Edna Moda, diseñadora de ropa de lujo para superhéroes, necesita una herramienta para gestionar las citas en su taller de diseño de manera eficiente. De esta necesidad surge AntiCape-Corp y su software gestor de citas. La aplicación tiene como objetivo principal facilitar la administración de clientes, trajes, empleados, talleres y reservas. De esta manera, cada cita se asigna correctamente según la disponibilidad y requisitos del traje. La aplicación es desarrollada en el lenguaje de programación Java, utilizando el entorno de desarrollo Eclipse. Esto se conectará a una base de datos MySQL para almacenar toda la información de manera organizada. La interfaz gráfica será fácil de usar e intuitiva, para que los empleados del taller puedan utilizarla sin dificultad; cada empleado solo podrá acceder a las funciones que le corresponden según su rol.

Para desarrollar la aplicación, el equipo de trabajo utilizará la metodología ágil SCRUM. Esto permite organizar las tareas en ciclos cortos llamados sprints, que en el contexto de este proyecto tienen una duración aproximada de dos semanas. En cada sprint, se abordarán un conjunto de funcionalidades concretas que le dan sentido a la aplicación en cada fase de su desarrollo.

A lo largo del proceso, se prestará especial atención a la calidad del programa. Se realizarán pruebas periódicas y se documentará el código utilizando JavaDoc. También se mantendrá un repositorio en la plataforma de GitHub que refleje la evolución de nuestro trabajo.

AntiCape-Corp busca cumplir con los requerimientos de Edna Moda y ofrecer una experiencia de usuario satisfactoria, sentando las bases para un desarrollo ordenado y profesional, acorde con las competencias adquiridas en el ciclo formativo de Desarrollo de Aplicaciones Web.

2. Objetivos

2.1. Generales

1. Desarrollar una aplicación funcional de gestión de citas que permita administrar clientes, trajes, empleados, talleres y reservas, cumpliendo con todos los requisitos funcionales que conlleva el desarrollo.
2. Aplicar una metodología de trabajo ágil SCRUM para gestionar el ciclo de vida del software, fomentando la colaboración en equipo, la adaptación a los cambios y la entrega continua a lo largo de los diferentes sprints del proyecto.

2.2. Específicos

1. Diseñar una base de datos que almacene la información de clientes, trajes, empleados, talleres y citas, estableciendo las relaciones y restricciones necesarias para garantizar la integridad de los datos.
2. Implementar interfaces de login que autentique a los empleados y establezca su categoría con el fin de controlar los permisos de acceso a las diferentes funcionalidades de la aplicación.
3. Programar las operaciones CRUD (Crear, Leer, Actualizar, Borrar) completas para la gestión de clientes y trajes, restringiendo el acceso de modificación y borrado en base a las categorías establecidas para los empleados.
4. Desarrollar la lógica de negocio para la creación y modificación de citas, validando que los empleados asignados tengan la categoría adecuada y que el taller esté disponible en la fecha y hora seleccionadas.
5. Configurar el sistema de permisos para que los oficiales solo puedan modificar sus propias citas y crear nuevas, mientras que los maestros tengan control total sobre todas las citas, clientes y talleres.

6. Crear la interfaz gráfica de usuario siguiendo el patrón Modelo Vista Controlador, que permita una navegación intuitiva y muestre la información de manera clara, diferenciando visualmente las opciones disponibles según el rol del empleado.
7. Generar la documentación técnica del proyecto utilizando JavaDoc con el fin de explicar el funcionamiento de la aplicación para cada tipo de usuario y colaborador en el proyecto.
8. Elaborar diagramas UML para modelar el comportamiento y la estructura de la aplicación, sirviendo como guía para la fase de implementación.
9. Gestionar el código fuente del proyecto en un repositorio GitHub, utilizando ramas para el desarrollo de funcionalidades, realizando commits frecuentes y documentando el progreso a través de tableros colaborativos como Trello.

3. Tecnologías utilizadas

La elaboración del proyecto abarcó áreas y tecnologías que van desde la parte más práctica, como la programación orientada a objetos, las bases de datos SQL y la creación de interfaces gráficas, hasta la parte más teórica, como la elaboración de diagramas MER, UML y Gantt. Estos últimos fueron clave para establecer las bases conceptuales sobre el funcionamiento interno de la gestión de citas, clientes, talleres y empleados; las tecnologías utilizadas para trabajar en las áreas anteriormente mencionadas fueron:

Programación:

Java (JDK): Lenguaje de programación principal para el desarrollo de la lógica y la arquitectura orientada a objetos.

Eclipse IDE: Entorno de Desarrollo Integrado utilizado para escribir, compilar y probar el código.

Gestión de Base de Datos:

MySQL Server & Workbench: Sistema gestor utilizado para la creación y administración de la base de datos relacional, así como para la manipulación visual de las tablas y consultas.

Arquitectura del proyecto:

WireFrameSketcher: Plugin integrado en Eclipse utilizado para el diseño, maquetado y prototipado de las ventanas de la interfaz gráfica.

Draw.io: Herramienta gráfica empleada para la elaboración de los diagramas estructurales (Modelo Entidad/Relación, Casos de Uso, Diagrama de Clases).

Gestión del Proyecto y Control de Versiones:

Trello: Herramienta basada en tableros Kanban para la planificación ágil y el seguimiento de las tareas de los Sprints.

Online Gantt (Web): Aplicación web utilizada para generar el diagrama de Gantt, facilitando la secuenciación temporal y visual del proyecto.

GitHub: Plataforma de alojamiento para el control de versiones del código fuente y el trabajo colaborativo del equipo

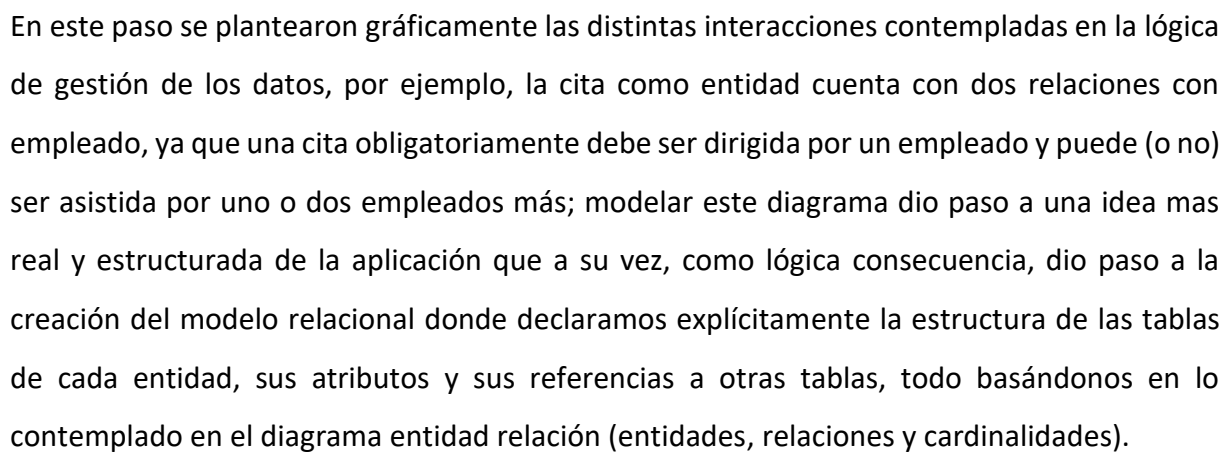
4. Desarrollo e implementación

La aplicación gestora, desarrollada por AntiCape-Corp, nace de la necesidad de Edna Moda, diseñadora de trajes para superhéroes, quien requiere una herramienta para administrar las citas en sus talleres de manera eficiente entre otras funciones.

La aplicación está orientada exclusivamente a los empleados del taller, los cuales se dividen en tres categorías: aprendices, oficiales y maestros. Los aprendices solo pueden visionar las citas. Los oficiales pueden crear nuevas citas y modificar únicamente aquellas que tienen asignadas a su nombre. Los maestros disponen de control total sobre todas las citas, clientes, trajes y talleres, pudiendo realizar cualquier operación de creación, lectura, modificación o eliminación.

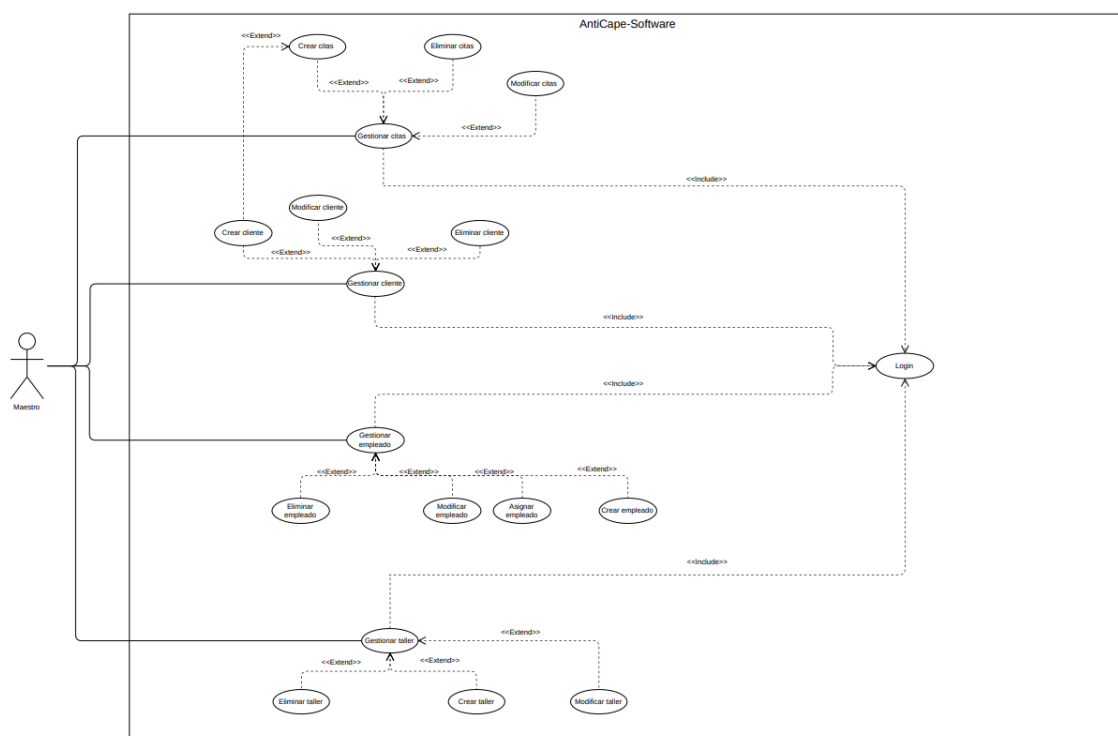
Entre las funcionalidades concretas se contemplan: el registro y modificación de clientes junto con sus trajes; la asignación de citas a talleres en una fecha y hora determinadas; la validación automática de disponibilidad del taller; la comprobación de la categoría del empleado asignado; y la consulta de todas las citas registradas. Todo ello se presenta a través de una interfaz gráfica que adapta las opciones visibles según el rol del empleado autenticado.

Tras el análisis previo, se dio paso al inicio del desarrollo del proyecto, el primer gran reto se dio en la planificación estructural de como queríamos que nuestra aplicación gestionara la lógica de creación, modificación y eliminación de todo lo que involucra el diseño de trajes (citas, clientes, empleados y talleres); para la planificación y aterrizado de todos estos detalles se decidió iniciar por lo mas importante, la base de datos, estableciendo que entidades interactúan en la gestión y como lo hacen, por lo que el primer paso lógico fue la creación del diagrama Entidad relación.



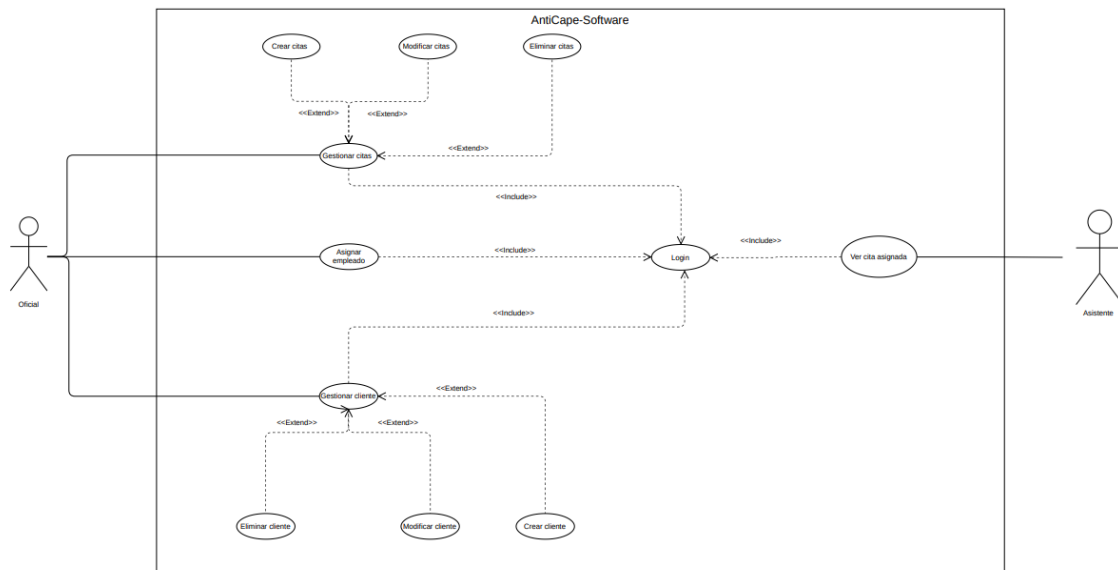
Con el diagrama del modelo relacional ya existe un acercamiento directo a la creación de la base de datos en MYSQL Workbench, pero en este punto consideramos que no se tuvo toda la arquitectura de lógica necesaria para iniciar con la declaración de creación de tablas.

Una de las faltas evidenciadas fue saber el como el usuario final interactuaría con la aplicación y por consiguiente con la base de datos, para modelar esto la herramienta utilizada fue el diagrama de casos de uso, donde se contempla que actores usaran la aplicación y que características se espera que puedan utilizar, esto tomando en cuenta el planteamiento de las distintas categorías que un empleado puede tener



Este primer diagrama modela el uso del actor con mayor poder dentro de la aplicación, el empleado de categoría **Maestro**. Este actor engloba prácticamente la totalidad de las funciones que ofrece el *AntiCape-software*, así como los posibles escenarios en los que dicho empleado accedería a funcionalidades específicas del programa.

Dado su rol privilegiado, el Maestro no solo tiene acceso a todas las operaciones de gestión, sino que también actúa como administrador de referencia dentro del flujo de trabajo. Por ejemplo, un empleado Maestro puede necesitar utilizar la aplicación para crear un nuevo registro de cliente. Sin embargo, como paso previo obligatorio, debe autenticarse en el sistema. Esta condición se encuentra modelada mediante una relación de tipo «include» entre el caso de uso *Login* y el resto de las gestiones.



En este otro diagrama de casos de uso, representamos a los restantes tipos de empleado, el **oficial** y el **aprendiz**, cuyas funcionalidades se encuentran limitadas en función de su rol.

- El **oficial** puede únicamente gestionar clientes, gestionar citas y asignar empleados a dichas citas. Su nivel de permisos es intermedio, ya que no dispone de las capacidades completas del Maestro, pero sí de las herramientas operativas esenciales para el día a día del taller o servicio.
- El **aprendiz**, por su parte, cuenta con un conjunto de permisos muy restringido, únicamente puede visualizar las citas a las que ha sido asignado, sin posibilidad de crearlas, modificarlas ni eliminarlas.

Al igual que en el diagrama anterior, se repite la relación de tipo «**include**» con el caso de uso *Login*, de manera que tanto oficial como aprendiz deben autenticarse obligatoriamente antes de acceder a cualquier funcionalidad. Adicionalmente, se modelan relaciones de tipo «**extend**» que engloban todas las operaciones asociadas a la gestión completa de cada elemento es decir, creación, modificación y eliminado.

Estas extensiones permiten representar que, dependiendo del tipo de empleado, ciertas operaciones estarán disponibles o no dentro del flujo principal de "Gestionar X".

Ya contando con la lógica de interacciones con el usuario y con una comprensión clara de cómo los datos serán manejados en la base de datos, nos pusimos manos a la obra con el desarrollo de la base de datos en MySQL Workbench.

Lo primero que hicimos fue generar un script con la declaración de todas las tablas del sistema, contemplando sus respectivas claves primarias, claves foráneas y tipos de datos. Este script inicial nos sirvió como esqueleto funcional del modelo relacional.

La generación anticipada del script resultó de gran utilidad, ya que nos permitió contar con un ejemplo directo y ejecutable de la **integridad referencial** de los datos. Es decir, pudimos comprobar de forma temprana que las relaciones entre tablas (como las existentes entre *empleados*, *clientes*, *citas* y *asignaciones*) respetan las reglas de negocio definidas previamente, evitando huérfanos o inconsistencias.

Además, este enfoque nos ayudó a validar el comportamiento de nuestro sistema de operaciones **CRUD** (Crear, Leer, Actualizar, Eliminar). Al disponer de un esquema real con integridad referencial activa, cada operación de inserción, modificación o eliminación dispara las restricciones, lo que nos permitió ajustar la lógica de la aplicación antes de avanzar a fases de programación.

```

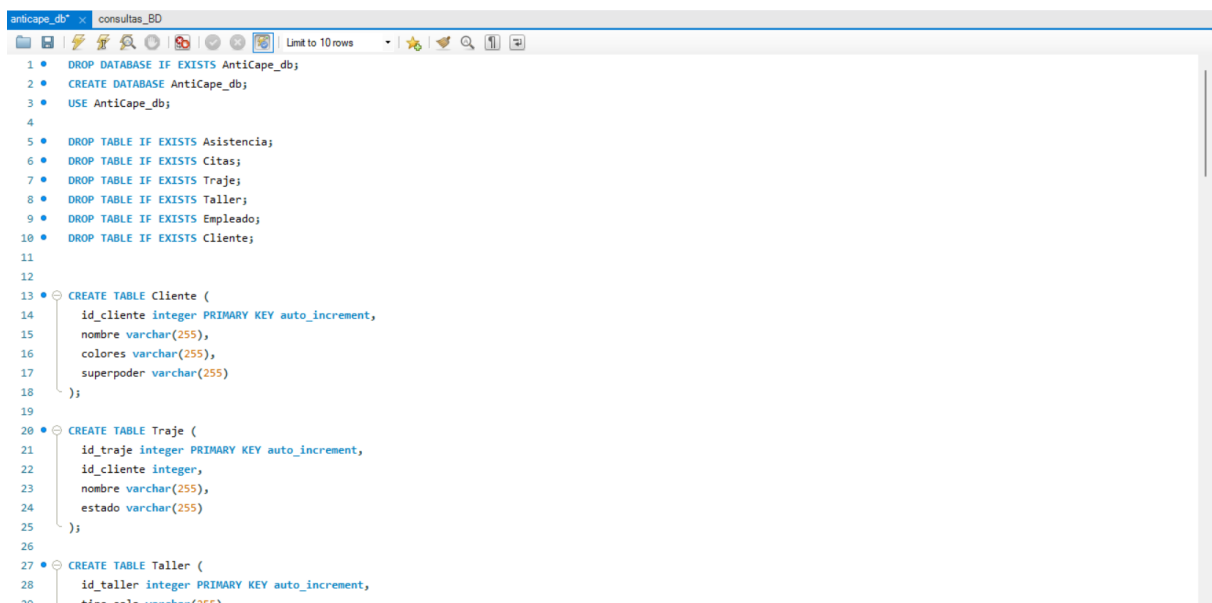
Table-creation
1 CREATE TABLE Cliente (
2     id_cliente integer PRIMARY KEY,
3     nombre varchar(255),
4     colores varchar(255),
5     superpoder varchar(255)
6 );
7
8 CREATE TABLE Traje (
9     id_traje integer PRIMARY KEY,
10    id_cliente integer,
11    nombre varchar(255),
12    estado varchar(255)
13 );
14
15 CREATE TABLE Taller (
16     id_taller integer PRIMARY KEY,
17     tipo_sala varchar(255),
18     nombre_sala varchar(255)
19 );
20
21 CREATE TABLE Empleado (
22     id_empleado integer PRIMARY KEY,
23     nombre varchar(255),
24     apellidos varchar(255),
25     apodo varchar(255),
26     categoria varchar(255),
27     contraseña varchar(255)
28 );
29
    
```

Paralelo a esto, desarrollamos otro script con datos iniciales de ejemplo, con el fin de contar con un punto de partida sólido para las operaciones del sistema, la estrategia fue la siguiente: una vez ejecutado el script de creación de tablas, lanzamos un segundo conjunto de instrucciones INSERT que introducían registros ficticios pero coherentes en todas las tablas, empleados de distintos roles, clientes de ejemplo, citas programadas en diferentes fechas y estados, así como las correspondientes asignaciones entre aprendices y citas.

```

datos_iniciales
7 INSERT INTO Cliente (id_cliente, nombre, colores, superpoder) VALUES(007, 'Thor', 'Plateado', 'Control del rayo y fuerza divina');
8 INSERT INTO Cliente (id_cliente, nombre, colores, superpoder) VALUES(008, 'ElasticGirl', 'Rojo', 'Estirar su cuerpo y cambiar de forma');
9 INSERT INTO Cliente (id_cliente, nombre, colores, superpoder) VALUES(009, 'Violet', 'Morado', 'Volverse invisible y crear campos de fuerza');
10 INSERT INTO Cliente (id_cliente, nombre, colores, superpoder) VALUES(010, 'Capitana Marvel', 'Rojo', 'Volar, absorber energía y súper fuerza');
11
12 INSERT INTO Traje(id_traje, nombre, estado) VALUES(001, 'principal', 'taller');
13 INSERT INTO Traje(id_traje, nombre, estado) VALUES(002, 'especifico', 'diseño');
14 INSERT INTO Traje(id_traje, nombre, estado) VALUES(003, 'principal', 'costura');
15 INSERT INTO Traje(id_traje, nombre, estado) VALUES(004, 'especifico', 'taller');
16 INSERT INTO Traje(id_traje, nombre, estado) VALUES(005, 'principal', 'diseño');
17 INSERT INTO Traje(id_traje, nombre, estado) VALUES(006, 'especifico', 'costura');
18 INSERT INTO Traje(id_traje, nombre, estado) VALUES(007, 'principal', 'taller');
19 INSERT INTO Traje(id_traje, nombre, estado) VALUES(008, 'especifico', 'diseño');
20 INSERT INTO Traje(id_traje, nombre, estado) VALUES(009, 'principal', 'costura');
21 INSERT INTO Traje(id_traje, nombre, estado) VALUES(010, 'especifico', 'taller');
22 INSERT INTO Traje(id_traje, nombre, estado) VALUES(011, 'principal', 'diseño');
23 INSERT INTO Traje(id_traje, nombre, estado) VALUES(012, 'especifico', 'costura');
24 INSERT INTO Traje(id_traje, nombre, estado) VALUES(013, 'principal', 'taller');
25 INSERT INTO Traje(id_traje, nombre, estado) VALUES(014, 'especifico', 'diseño');
26 INSERT INTO Traje(id_traje, nombre, estado) VALUES(015, 'principal', 'costura');
27 INSERT INTO Traje(id_traje, nombre, estado) VALUES(016, 'especifico', 'taller');
28 INSERT INTO Traje(id_traje, nombre, estado) VALUES(017, 'principal', 'diseño');
29 INSERT INTO Traje(id_traje, nombre, estado) VALUES(018, 'especifico', 'costura');
30 INSERT INTO Traje(id_traje, nombre, estado) VALUES(019, 'principal', 'taller');
31 INSERT INTO Traje(id_traje, nombre, estado) VALUES(020, 'especifico', 'diseño');
32
33
34 INSERT INTO Taller(id_taller, tipo_sala, nombre_sala) VALUES (001, 'Paris', 'diseño');
35 INSERT INTO Taller(id_taller, tipo_sala, nombre_sala) VALUES (002, 'Nueva York', 'diseño');
    
```

Una vez generados por separado el script de creación de tablas y el script de inserción de datos de ejemplo, se procedió a integrar ambos en un único fichero de despliegue. Este nuevo script unificado incorporaba, además de las sentencias CREATE TABLE e INSERT, la sintaxis completa para la creación y selección de la base de datos (CREATE DATABASE IF NOT EXISTS y USE), garantizando así que todo el entorno relacional pudiera levantarse de forma autónoma y reproducible. De esta manera, bastaba con ejecutar un solo bloque de instrucciones en para disponer de la estructura y los datos iniciales necesarios, simplificando la puesta en marcha del sistema y facilitando las pruebas de integración de las operaciones CRUD.



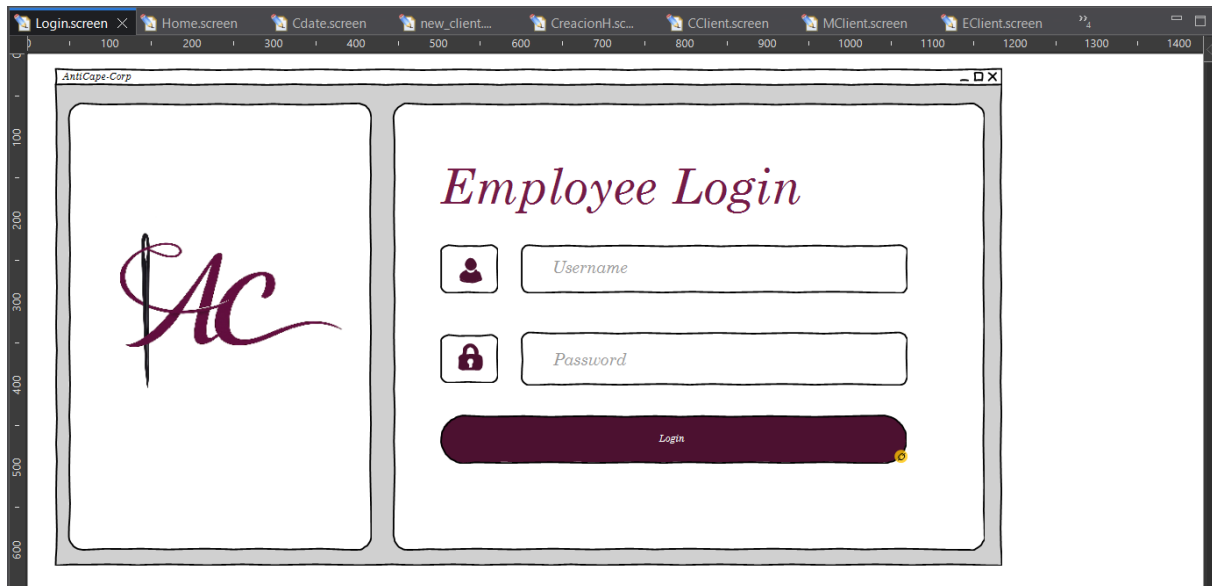
```

1 DROP DATABASE IF EXISTS AntiCape_db;
2 CREATE DATABASE AntiCape_db;
3 USE AntiCape_db;
4
5 DROP TABLE IF EXISTS Asistencia;
6 DROP TABLE IF EXISTS Citas;
7 DROP TABLE IF EXISTS Traje;
8 DROP TABLE IF EXISTS Taller;
9 DROP TABLE IF EXISTS Empleado;
10 DROP TABLE IF EXISTS Cliente;
11
12
13 CREATE TABLE Cliente (
14     id_cliente integer PRIMARY KEY auto_increment,
15     nombre varchar(255),
16     colores varchar(255),
17     superpoder varchar(255)
18 );
19
20 CREATE TABLE Traje (
21     id_traje integer PRIMARY KEY auto_increment,
22     id_cliente integer,
23     nombre varchar(255),
24     estado varchar(255)
25 );
26
27 CREATE TABLE Taller (
28     id_taller integer PRIMARY KEY auto_increment,
29     direccion varchar(255),

```

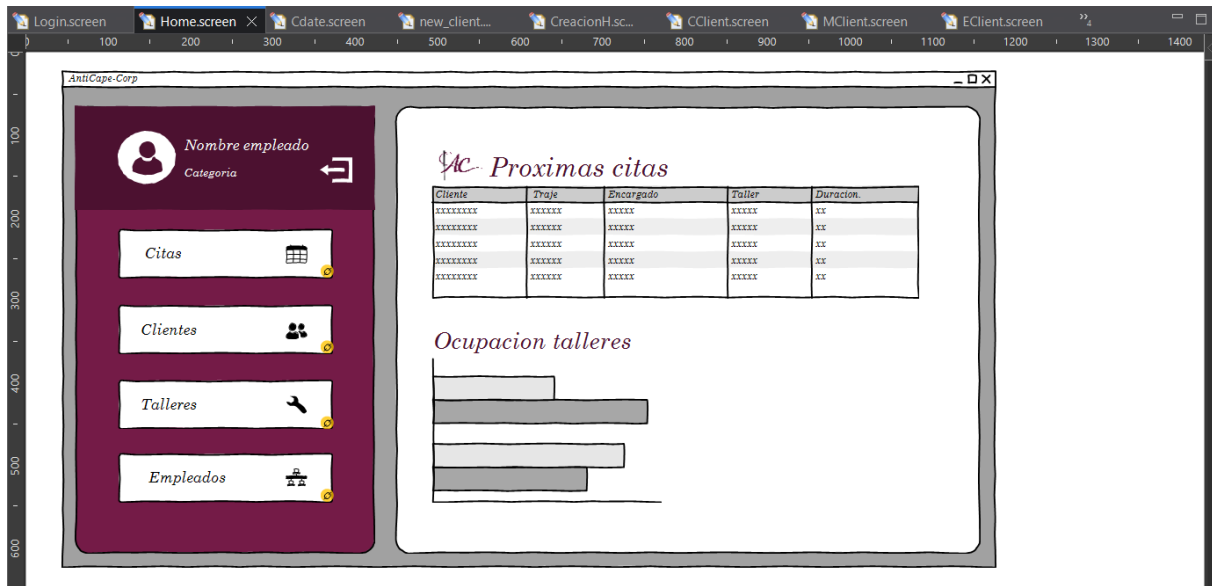
Una vez consolidada toda la lógica subyacente de la base de datos y validado su correcto funcionamiento, el equipo se centró en el diseño de los wireframes de la aplicación. Este proceso se abordó con el propósito de trasladar a la interfaz gráfica los valores de elegancia y estatus que definen la identidad de *AntiCape-Corp*. Para ello, se empleó la herramienta WireFrameSketcher integrada en Eclipse, buscando plasmar una estética coherente con el branding desarrollado; predominio del tono burdeos corporativo (Pantone 19-1617 TCX), la tipografía cuidada y una disposición de elementos que transmitiese sobriedad y distinción sin renunciar a la claridad informativa. Se puso especial énfasis en lograr una *navegación* intuitiva y accesible, donde cada ventana, botón y campo de texto estuviese ubicado de forma lógica y predecible.

Además, se prestó una atención minuciosa a los detalles más sutiles, como la alineación de los componentes, el espaciado entre elementos y la retroalimentación visual ante las acciones del usuario, con el objetivo de ofrecer una experiencia de uso fluida, profesional y acorde con el nivel de excelencia que Edna Moda exige en su taller.



Durante el proceso de esquematización, se optó por una estructura de interfaz basada en un panel de navegación lateral izquierdo de carácter estático, mientras que el resto de la ventana se reservaba para la visualización dinámica de los diferentes módulos de gestión.

Esta decisión de diseño respondió a la necesidad de ofrecer al empleado un punto de referencia **constante e inmutable**, independientemente de la pantalla en la que se encontrase. Los botones del panel lateral se dispusieron siguiendo un orden jerárquico estricto, atendiendo a la relevancia y frecuencia de uso de cada funcionalidad dentro del flujo de trabajo diario del taller. Así, en la posición más privilegiada se situó el acceso a la tabla de citas que es considerada el núcleo operativo de la aplicación, seguida de los accesos a la gestión de clientes, talleres y empleados, en ese orden descendente. Por último, en la parte superior del panel se ubicaron las opciones auxiliares, como el perfil del empleado autenticado y el botón de cierre de sesión. Esta disposición permitió que, de un solo vistazo, el usuario identificase las acciones principales y navegase de forma ágil e intuitiva, manteniendo siempre a la vista su nombre y categoría, elementos clave para recordar sus permisos activos mientras el área central se actualizaba con la tabla de clientes, la ocupación de talleres o cualquier otra vista seleccionada.



Dentro de cada área de gestión ya fuese citas, clientes, talleres o empleados, se dispuso un menú superior derecho específico que concentra todas las operaciones CRUD asociadas a dicho módulo. Este menú, incluye botones claramente etiquetados para las acciones de creación, modificación y eliminación, permitiendo al empleado ejecutar cualquier operación sin necesidad de abandonar la vista actual.

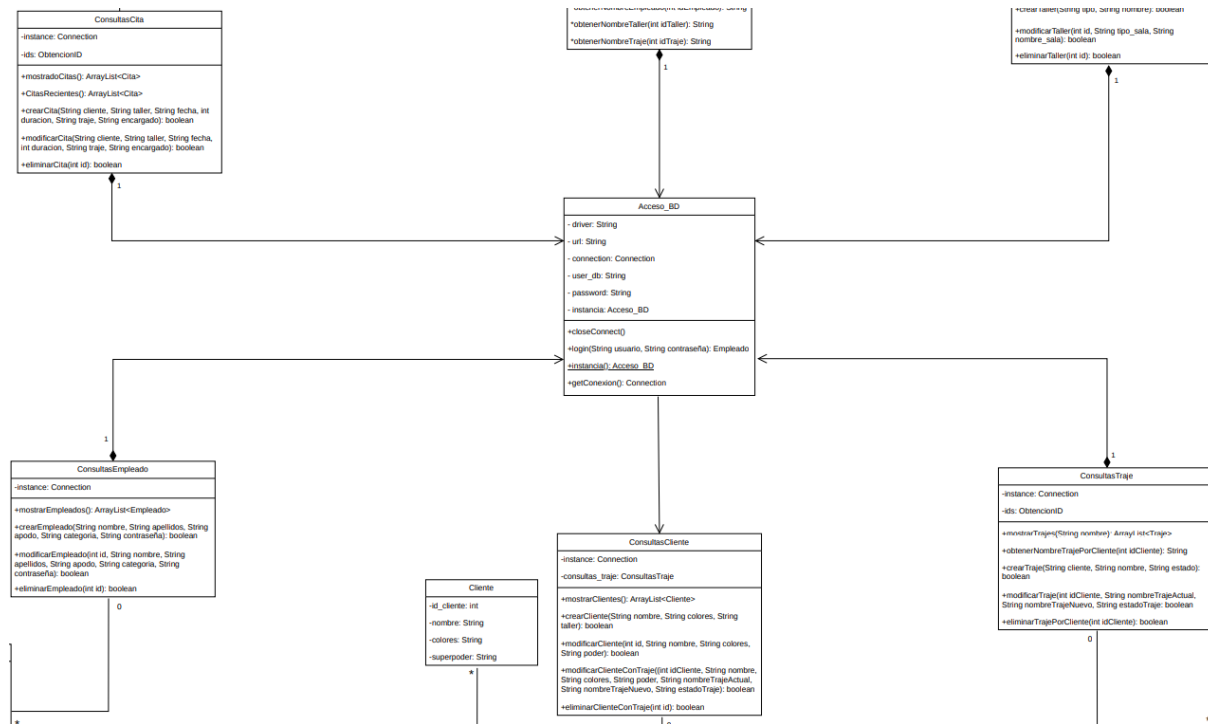
La ubicación fija de estos controles en la parte superior derecha respondió a los principios de la ley de Fitts y a los patrones de navegación más extendidos en aplicaciones de escritorio, facilitando así un acceso rápido e instintivo. Complementariamente, en el área central de la ventana se habilitó un pequeño "home interno" que mostraba, a modo de tabla resumen, la generalidad de todos los registros pertenecientes a dicho apartado. Por ejemplo, al acceder al módulo de talleres, el usuario visualizaba de forma inmediata un listado completo con todos los talleres registrados incluyendo su tipo (Diseño, Costura o Pruebas) y el nombre de la sala (Berlín, Tokio, Madrid, Milán o París), pudiendo filtrar, ordenar o seleccionar directamente cualquier registro para proceder a su modificación o eliminación mediante el menú superior. Esta combinación de menú y vista permitió que el empleado tuviese siempre una visión global del estado del sistema a la vez que accedía con un solo clic a las operaciones más detalladas, optimizando así la eficiencia y reduciendo la curva de aprendizaje y familiarización con el programa y todas sus funcionalidades.

Una vez finalizada la fase de prototipado de los wireframes y validadas las decisiones de interfaz con el resto del equipo, se dio por concluida la etapa de diseño visual de la aplicación. Los wireframes obtenidos sirvieron como guía fidedigna no solo para la implementación posterior de las ventanas, sino también para comprender los flujos de navegación y las interacciones esperadas por parte del usuario.

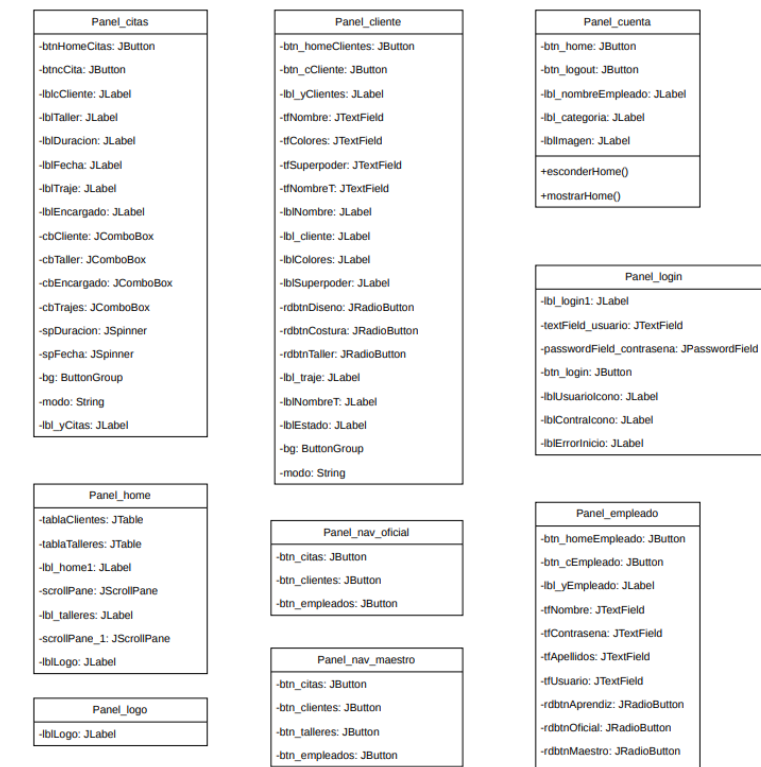


Acto seguido, el equipo se dispuso a abordar el siguiente paso fundamental del desarrollo, la elaboración del diagrama de clases, tomando como base arquitectónica el patrón **Modelo-Vista-Controlador (MVC)**. Este patrón fue seleccionado por su capacidad para separar de manera limpia las distintas capas de la aplicación, facilitando así el mantenimiento, las pruebas y la escalabilidad futura del software.

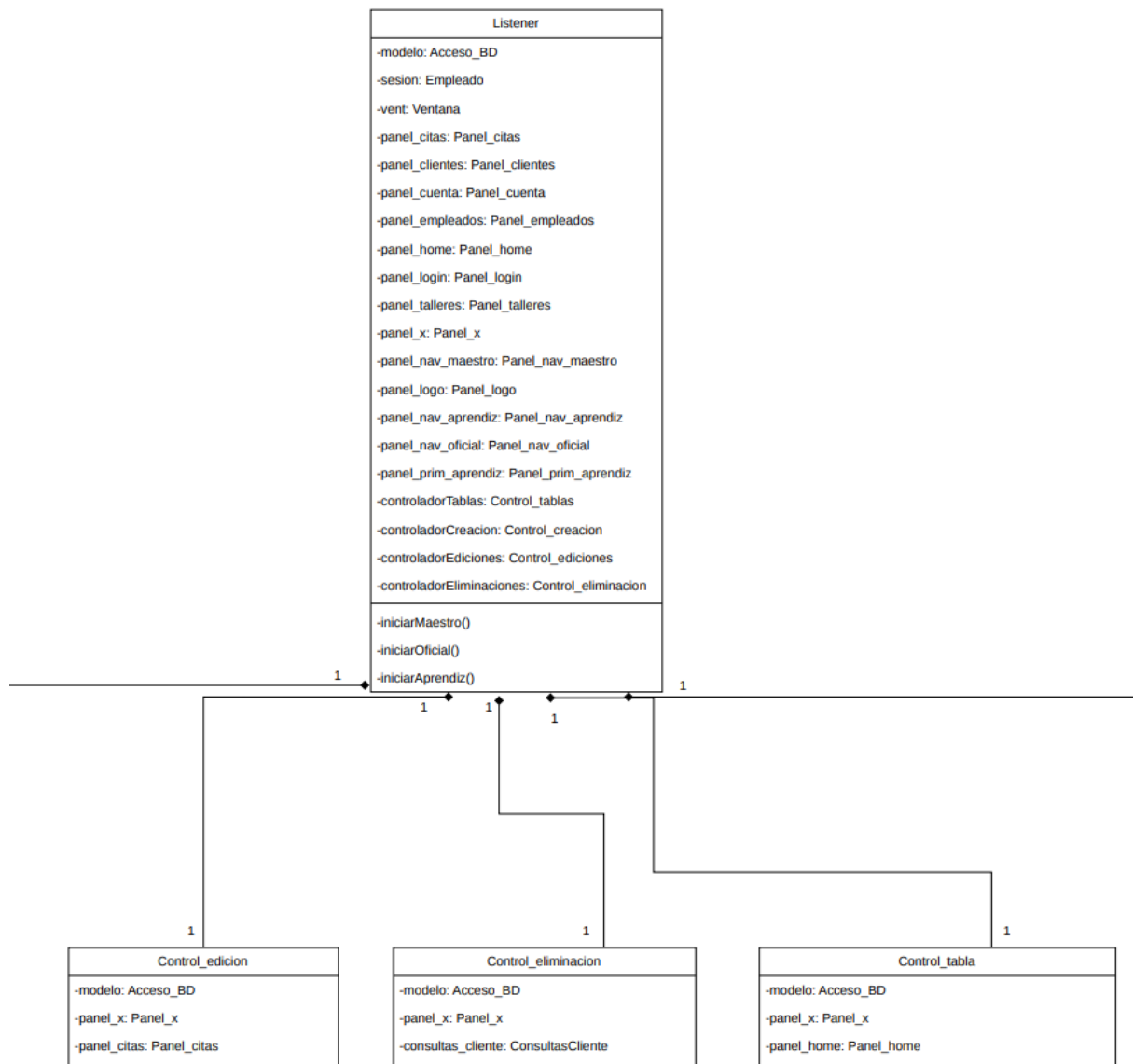
En concreto, se definió que la capa **Modelo** contendría toda la lógica de conexión y consulta a la base de datos MySQL, encapsulando las operaciones CRUD y la validación de disponibilidad de talleres, sin que el resto de las capas dependiese directamente de la tecnología de persistencia empleada.



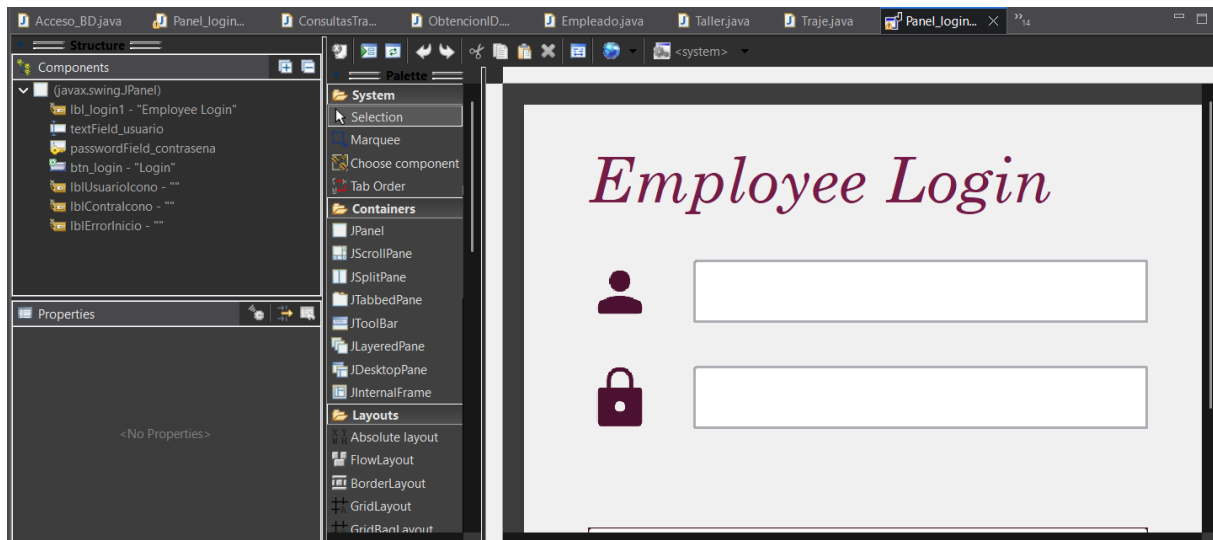
La capa **Vista** se implementaría íntegramente con Java Swing, plasmando fielmente los wireframes diseñados, adaptando dinámicamente los elementos visibles en función de la categoría del empleado autenticado.



Por su parte, la capa **Controlador** actuaría como puente entre ambas, albergando el poder operativo de la aplicación: gestionaría los eventos disparados por la interfaz, orquestaría las peticiones al modelo y actualizaría la vista con los resultados obtenidos. Esta separación de responsabilidades permitió que cada miembro del equipo pudiese trabajar de forma paralela en distintas capas, siempre bajo el paraguas de un diagrama de clases común que reflejaba con precisión la estructura estática del sistema. El diagrama de clases, por tanto, se convirtió en el plano arquitectónico definitivo previo a la fase de codificación, asegurando la coherencia entre el diseño conceptual, la base de datos y la futura implementación en Java.



Una vez completado el modelado de clases en el diagrama UML bajo el patrón Modelo-Vista-Controlador, el equipo inició la fase de codificación. El primer punto abordado fue la implementación de la capa Vista, para lo cual se empleó la herramienta WindowBuilder integrada en Eclipse, que permitió agilizar notablemente el maquetado de las ventanas mediante un editor visual de interfaces Swing.



No obstante, el equipo optó por complementar esta aproximación visual con ajustes manuales precisos en el código generado, garantizando así un control total sobre cada componente y su disposición. El primer panel desarrollado fue el Panel_login, encargado de la autenticación de los empleados. En este panel pueden observarse varias decisiones de diseño que reflejan el cuidado por los detalles y la identidad corporativa.

Se utilizó un layout absoluto (setLayout(null)) para posicionar cada elemento con coordenadas exactas, lo que permitió un control milimétrico sobre la alineación y el espaciado.

```
public class Panel_login extends JPanel {

    private JLabel lbl_login1;
    private JTextField textField_usuario;
    private JPasswordField passwordField_contrasena;
    private JButton btn_login;
    private JLabel lblUsuarioIcono;
    private JLabel lblContraIcono;
    private JLabel lblErrorInicio;

    public Panel_login(Listener list) {
        setLayout(null);
        setSize(723, 545);

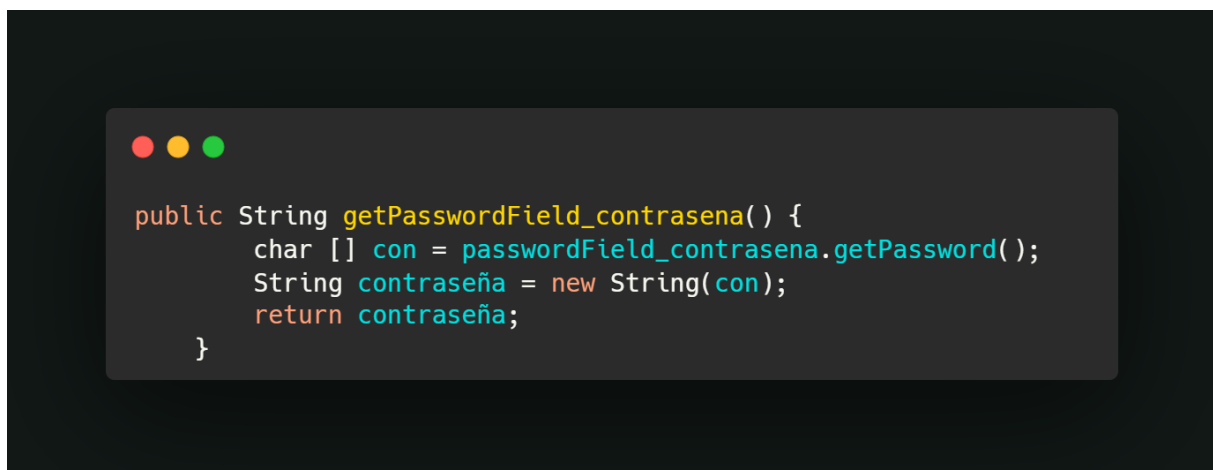
        lbl_login1 = new JLabel("Employee Login");
        lbl_login1.setForeground(new Color(116, 27, 71));
        lbl_login1.setFont(new Font("Century Schoolbook",
        Font.ITALIC, 60));
        lbl_login1.setBounds(61, 24, 579, 88);
        add(lbl_login1);

        textField_usuario = new JTextField();
        textField_usuario.setFont(new Font("Century Schoolbook",
        Font.PLAIN, 30));
        textField_usuario.setBorder(new LineBorder(new Color(171,
        173, 179), 3, true));
        textField_usuario.setToolTipText("usuario");
        textField_usuario.setBounds(161, 149, 432, 60);
        add(textField_usuario);
```

La tipografía elegida fue "Century Schoolbook" en diferentes pesos y tamaños —desde 15 puntos para el botón hasta 60 puntos para el título—, otorgando un carácter distinguido y académico que armoniza con la imagen de lujo y elegancia de AntiCape-Corp.

Los campos de texto y contraseña incorporaron bordes redondeados personalizados mediante LineBorder con un grosor de 3 píxeles y un color gris neutro (#ABADB3), logrando un acabado moderno y profesional. Destaca especialmente la inclusión de iconos gráficos (lblUsuarioIcono y lblContraIcono), que refuerzan la usabilidad al aportar una referencia visual inmediata a cada campo. El botón de inicio de sesión, con fondo de color burdeos corporativo (RGB: 76, 17, 48) y texto blanco, actúa como elemento focal de la ventana. Asimismo, se reservó un JLabel específico (lblErrorInicio) para mostrar mensajes de error en rojo y cursiva ante credenciales inválidas, mejorando la retroalimentación al usuario.

La lógica de extracción de la contraseña se encapsuló en un método específico (`getPasswordField_contrasena`) que convierte el array de caracteres devuelto por `JPasswordField` en un objeto `String`, siguiendo las buenas prácticas de seguridad al no almacenar la contraseña en un `String` más tiempo del necesario. De esta manera, el `Panel_login` sentó las bases visuales y funcionales sobre las que se construiría el resto de la interfaz, demostrando una integración coherente entre el diseño previo en wireframes y la implementación real en Java Swing.

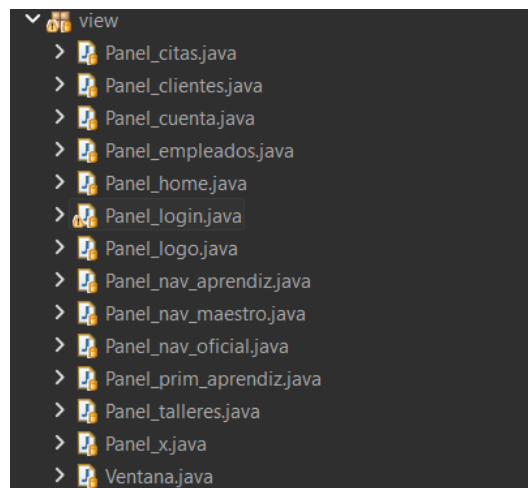


```

public String getPasswordField_contrasena() {
    char [] con = passwordField_contrasena.getPassword();
    String contraseña = new String(con);
    return contraseña;
}

```

A partir del **Panel_login** como punto de entrada, se desarrollaron el resto de los paneles que conforman la capa de Vista de la aplicación, siguiendo una estructura homogénea y coherente con los wireframes previamente diseñados. El paquete `view` quedó compuesto por un total de trece clases, entre las que destacan los paneles específicos para cada módulo de gestión, **Panel_citas**, **Panel_clientes**, **Panel_empleados** y **Panel_talleres**, los paneles de navegación diferenciados por rol, **Panel_nav_aprendiz**, **Panel_nav_oficial** y **Panel_nav_maestro**, y paneles auxiliares como **Panel_home**, **Panel_cuenta** y **Panel_logo**.



```

view
├── Panel_citas.java
├── Panel_clientes.java
├── Panel_cuenta.java
├── Panel_empleados.java
├── Panel_home.java
├── Panel_login.java
├── Panel_logo.java
├── Panel_nav_aprendiz.java
├── Panel_nav_maestro.java
├── Panel_nav_oficial.java
├── Panel_prim_aprendiz.java
├── Panel_talleres.java
├── Panel_x.java
└── Ventana.java

```

Adicionalmente, la clase Ventana actuó como contenedor principal, orquestando el ensamblaje dinámico de los distintos paneles en función de la navegación del usuario.

Todos los paneles heredaron de JPanel y adoptaron una serie de convenciones comunes; uso de layout absoluto para un control exhaustivo del posicionamiento, dimensionado fijo de 723x545 píxeles para garantizar la consistencia visual, e implementación de constructores que reciben como parámetro un objeto Listener procedente de la capa de control, estableciendo así el puente de comunicación entre la Vista y el Controlador.

Como ejemplo ilustrativo de esta arquitectura, el Panel_home que funciona como dashboard principal presenta dos tablas (JTable) encapsuladas en paneles de desplazamiento (JScrollPane) para mostrar las próximas citas y la ocupación de los talleres, junto con etiquetas que siguen la línea tipográfica y cromática definida, "Century Schoolbook" en cursiva y color burdeos corporativo. Además, incorpora un pequeño logotipo en la esquina superior izquierda, redimensionado mediante el método estático Ventana.escalarImagen(), que refuerza la identidad visual de AntiCape-Corp en cada pantalla.



Cada panel expone, a través de métodos getter públicos, sus componentes internos, tablas, botones y campos de texto permitiendo que el Controlador pueda leer los valores introducidos por el usuario y actualizar dinámicamente el contenido mostrado. De esta manera, la capa Vista se mantiene como una interfaz pasiva y declarativa, delegando toda la lógica operativa en el Controlador, en perfecta sintonía con el patrón MVC adoptado desde el modelado UML.

```

/**
 * @return the btn_homeEmpleado
 */
public JButton getBtn_homeEmpleado() {
    return btn_homeEmpleado;
}

public JButton getBtn_cEmpleado() {
    return btn_cEmpleado;
}

public JLabel getLbl_yEmpleado() {
    return lbl_yEmpleado;
}

public JTextField getTfNombre() {
    return tfNombre;
}

public JTextField getTfContrasena() {
    return tfContrasena;
}

```

Una vez finalizada la implementación de la capa Vista, ajustada fielmente a lo representado en los wireframes, el equipo se centró en el desarrollo de la capa Modelo, cuyo principal objetivo es establecer y gestionar la conexión con la base de datos MySQL. Para ello se creó la clase Acceso_BD dentro del paquete model, diseñada con una arquitectura que incorpora varias decisiones técnicas relevantes.

```

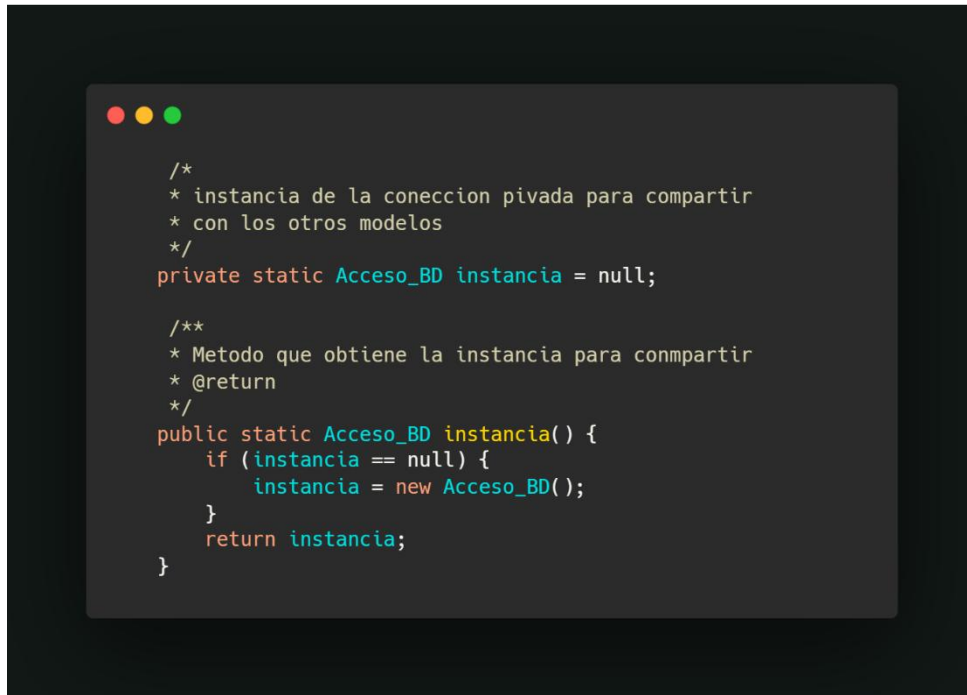
/**
 * @author Breixo García Canovas
 * @author Robinson Tamayo Guerrero
 * @author Romeo Rey Alonso
 * @author Sara Cardeña Carpio
 */
package model;

import java.sql.*;

/**
 * Clase que moldea el acceso a base de datos y todos sus metodos
 * de consultas
 */
public class Acceso_BD {
    private String driver = "com.mysql.cj.jdbc.Driver";
    private String url = "jdbc:mysql://localhost/AntiCape_db";
    private Connection connection = null;
    private String user_db = "root";
    private String password_db = "root";
}

```


En primer lugar, se implementó el patrón de diseño **Singleton**, garantizando que a lo largo de toda la ejecución de la aplicación exista una única instancia activa de la conexión a la base de datos. Esto se logró mediante un constructor privado (`private Acceso_BD()`) y un método estático `instancia ()` que devuelve siempre el mismo objeto, evitando así la creación innecesaria de múltiples conexiones concurrentes que podrían degradar el rendimiento o generar conflictos de recursos.



```
/*
 * instancia de la conexión privada para compartir
 * con los otros modelos
 */
private static Acceso_BD instancia = null;

/**
 * Metodo que obtiene la instancia para compartir
 * @return
 */
public static Acceso_BD instancia() {
    if (instancia == null) {
        instancia = new Acceso_BD();
    }
    return instancia;
}
```

Para la comunicación entre la aplicación Java y el servidor MySQL, se empleó el **conector JDBC** (`com.mysql.cj.jdbc.Driver`), correspondiente a la versión 8 del driver, que permite ejecutar sentencias SQL de forma nativa y segura. La URL de conexión apunta a la base de datos `AntiCape_db` alojada en `localhost`, y las credenciales de acceso se almacenan en variables privadas (`user_db` y `password_db`).

Como ejemplo concreto de operación sobre la base de datos, la clase incluye el método `login()`, que recibe usuario y contraseña, construye una consulta parametrizada mediante `PreparedStatement` previniendo así ataques de inyección SQL y devuelve un objeto `Empleado` con los datos recuperados si las credenciales son correctas, o `null` en caso contrario.

Cabe destacar el uso de la cláusula **try-with-resources** en los bloques de manejo de PreparedStatement y ResultSet, lo que garantiza el cierre automático de estos recursos al finalizar el bloque, mejorando notablemente la legibilidad y evitando fugas de memoria. De esta forma, la clase Acceso_BD se programó como el pilar del sistema, ofreciendo una interfaz limpia y segura al resto de componentes del Modelo.

```

/**
 * Metodo publico que devuelve un objeto de tipo empleado en base a su
 * coincidencia de usuario y contraseña
 *
 * @param usuario    apodo del empleado en la base de datos
 * @param contraseña contraseña del empleado en la base de datos
 * @return objeto de tipo empleado, null si no existe
 */
public Empleado login(String usuario, String contraseña) {
    System.out.println("metodo login");

    // consulta de usuario y contraseña unica
    String query = "SELECT id_empleado, nombre, apellidos, categoria FROM Empleado WHERE apodo = ? AND contraseña = ?";

    /*
     * NOTA: en todos los prepared statements, si se crea el statement en unos ()
     * la clausula try se encarga de abrir y cerrar el cursor mejorando la lectura y sintaxis del codigo
     */
    try (PreparedStatement stmt = connection.prepareStatement(query)) {

```

Para mantener una arquitectura limpia y fácilmente mantenible, el paquete model no se limitó únicamente a la clase de conexión, sino que se amplió con un conjunto de clases que cumplían dos propósitos fundamentales. Por un lado, se crearon clases que modelan directamente las entidades de la base de datos **Cita, Cliente, Empleado, Taller y Traje**, cada una de ellas con sus atributos, constructores y métodos getter/setter, actuando como **contenedores de datos** (POJOs) que facilitan el intercambio de información entre las distintas capas de la aplicación.

```

> model
  > Acceso_BD.java
  > Cita.java
  > Cliente.java
  > ConsultasCita.java
  > ConsultasCliente.java
  > ConsultasEmpleado.java
  > ConsultasTaller.java
  > ConsultasTraje.java
  > Empleado.java
  > ObtencionID.java
  > Taller.java
  > Traje.java

```

Por otro lado, se desarrollaron clases específicas para la ejecución de operaciones sobre cada entidad, siguiendo un patrón de nomenclatura claro, ConsultasCita , ConsultasCliente , ConsultasEmpleado, ConsultasTaller y ConsultasTraje.

Esta separación permitió que la lógica de acceso a datos estuviese perfectamente segmentada, delegando cada familia de operaciones a su clase correspondiente y evitando así la creación de una clase unica y difícil de depurar. Adicionalmente, la clase auxiliar ObtencionID centralizó los métodos de búsqueda de identificadores a partir de nombres, evitando duplicación de código y simplificando las consultas que requieran resolver claves foráneas.

```
public class ObtencionID {

    private Connection instance = null;

    public ObtencionID(Connection instance) {
        this.instance = instance;
    }

    /**
     * Metodo que obtiene el id de un cliente en base a su nombre
     *
     * @param nombre nombre del cliente
     * @return int con el valor del id del cliente (-1 en caso de que no exista el cliente)
     */
    protected int obtenerIdCliente(String nombre) {
        String query = "SELECT id_cliente FROM Cliente WHERE nombre = ?";
        try (PreparedStatement pstmt = instance.prepareStatement(query)) {
            pstmt.setString(1, nombre);
            ResultSet resultado = pstmt.executeQuery();
            if (resultado.next()) {
                return resultado.getInt("id_cliente");
            }
            resultado.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return -1;
    }

}
```

Como ejemplo representativo de esta arquitectura, la clase ConsultasCita recibe en su constructor un objeto Connection y una instancia de ObtencionID, estableciendo así las dependencias necesarias para operar sobre la base de datos.

El método `mostradoCitas()` ejecuta una consulta `SELECT * FROM Citas` mediante un **Statement** simple, recorriendo el **ResultSet** y construyendo objetos `Cita` que se añaden a un `ArrayList` para su posterior consumo por parte del Controlador y la Vista.

```
/**
 * Metodo publico que consulta a la base de datos todos los registros
 * de citas en el sistema
 *
 * @return ArrayList de objetos tipo cita
 */
public ArrayList<Cita> mostradoCitas() {
    ArrayList<Cita> citas = new ArrayList<>();
    String query = "SELECT * FROM Citas";

    try (Statement stmt = instance.createStatement(); ResultSet resultado =
        stmt.executeQuery(query)) {

        while (resultado.next()) {
            Cita cita = new Cita(resultado.getInt(1), resultado.getString(2),
                resultado.getString(3), resultado.getInt(4), resultado.getInt(5), resultado.getInt(6),
                resultado.getInt(7));
            citas.add(cita);
        }

        return citas;
    } catch (SQLException e) {
        e.printStackTrace();
        return new ArrayList<>();
    }
}
```

El método `CitasRecientes()` añade la cláusula `ORDER BY fecha ASC`, devolviendo las citas ordenadas de la más próxima a la más lejana, una funcionalidad clave para que el empleado pueda visualizar la agenda de forma priorizada.

```
/**
 * Metodo que consulta a la base de datos la totalidad
 * de citas en base a su fecha (de la mas proxima a la mas lejana)
 *
 * @return ArrayList de objetos de tipo cita
 */
public ArrayList<Cita> CitasRecientes() {
    ArrayList<Cita> citasRecientes = new ArrayList<>();
    String query = "SELECT * FROM Citas ORDER BY fecha ASC";

    try (Statement stmt = instance.createStatement(); ResultSet resultado = stmt.executeQuery(query)) {

        while (resultado.next()) {
            Cita cita = new Cita(resultado.getInt(1), resultado.getString(2), resultado.getString(3),
                resultado.getInt(4), resultado.getInt(5), resultado.getInt(6), resultado.getInt(7));
            citasRecientes.add(cita);
        }

        return citasRecientes;
    } catch (SQLException e) {
        e.printStackTrace();
        return new ArrayList<>();
    }
}
```

Para las operaciones de modificación de datos, se emplearon **PreparedStatement** en métodos como `crearCita()`, `modificarCita()` y `eliminarCita()`. Destacamos especialmente la lógica de `crearCita()`, que recibe parámetros en formato texto (nombre del cliente, nombre del taller, nombre del encargado y nombre del traje) y, mediante llamadas a `ObtencionID`, resuelve los id's necesarios para respetar las claves foráneas de la tabla Citas.

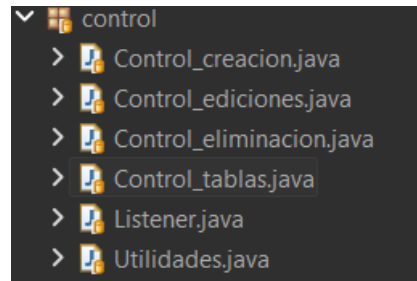
```
/**
 * Metodo de insercion de una nueva cita en la base de datos
 *
 * @param cliente Nombre del cliente de la cita
 * @param taller Nombre del taller donde se realizara la cita
 * @param fecha Fecha en la que se realizara la cita
 * @param duracion Duracion que tendra la cita ( en formato ejem: 1 H )
 * @param traje Nombre del traje que sera trabajado
 * @param encargado Nombre del empleado encargado de la cita
 *
 * @return true si se creo la cita, false si no
 */
public boolean crearCita(String cliente, String taller, String fecha, int duracion, String traje, String
encargado) {
    //probando un poco el prepared statement :)
    String query = "INSERT INTO Citas (fecha, duracion, id_cliente, id_encargado, id_taller, id_traje) VALUES
(?, ?, ?, ?, ?, ?)";

    try (PreparedStatement stmt = instance.prepareStatement(query)) {

        //Obtencion id del cliente por su nombre
        int idCliente = ids.obtenerIdCliente(cliente);
        if (idCliente == -1) {
            //prints de debug futuro
            System.out.println("Cliente no encontrado: " + cliente);
            return false;
        }
    }
}
```

Cada operación incluye mensajes de depuración en consola y retorna un valor booleano indicando el éxito o fracaso de la operación, lo que permite al controlador tomar decisiones informadas y mostrar retroalimentación al usuario. De esta manera, la capa Modelo quedó completamente desacoplada de la lógica de negocio y de la interfaz gráfica, cumpliendo con su responsabilidad exclusiva de gestionar la persistencia y ofrecer datos claros al resto del sistema.

Como último paso previo a disponer de una primera versión funcional de la aplicación, el equipo desarrolló el paquete más relevante desde el punto de vista funcional el paquete **control**, encargado de orquestar toda la lógica operativa del sistema.



Este paquete se estructuró en cinco clases especializadas que dividen las responsabilidades del CRUD de forma clara y estructurada:

- Control_creacion (gestiona la lógica de creación de nuevos registros)
- Control_ediciones (maneja las modificaciones)
- Control_eliminacion (coordina las operaciones de borrado)
- Control_tablas (actualiza dinámicamente las tablas de la interfaz)

y una clase auxiliar **Utilidades** que centraliza funciones comunes de validación y formateo de formularios. Sin embargo, el verdadero “núcleo neuronal” del sistema recae en la clase Listener, concebida como el "**comandante general**" de la aplicación.

```
package control;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.KeyEvent;

import model.Acceso_BD;
import model.Empleado;
import view.*;

public class Listener implements ActionListener {

    private Acceso_BD modelo = Acceso_BD.instancia();
    private Empleado sesion;

    private Ventana vent;
```

Esta clase implementa la interfaz ActionListener de Java Swing y actúa como el único punto de escucha para todos los eventos generados por la interfaz gráfica, desde clics en botones hasta cambios de estado en los formularios.

El **Listener** mantiene una referencia a la instancia Singleton del modelo (**Acceso_BD.instancia()**), almacena el objeto Empleado correspondiente a la sesión activa y contiene referencias a todos los paneles de la vista, que instancia directamente en su constructor.

```
Panel_talleres panel_talleres = new Panel_talleres(this);
Panel_x panel_x = new Panel_x(this);
Panel_nav_maestro panel_nav_maestro = new Panel_nav_maestro(this);
Panel_logo panel_logo = new Panel_logo(this);
Panel_nav_aprendiz panel_nav_aprendiz = new Panel_nav_aprendiz(this);
Panel_nav_oficial panel_nav_oficial = new Panel_nav_oficial(this);
Panel_prim_aprendiz panel_prim_aprendiz = new Panel_prim_aprendiz(this);

private Control_tablas controladorTablas = new Control_tablas(panel_x, panel_home);
private Control_creacion controladorCreacion = new Control_creacion(panel_clientes, panel_empleados, panel_citas, panel_talleres);
private Control_ediciones controladorEdiciones = new Control_ediciones(panel_x, panel_citas, panel_clientes, panel_empleados);
private Control Eliminacion controladorEliminaciones = new Control_eliminacion(panel_x);
```

Su método actionPerformed() actúa como un gran operador que, en función del origen del evento o del comando asociado a (.getActionCommand()), redirige el flujo de ejecución hacia la lógica correspondiente. Para mantener el código limpio y escalable, el Listener delega las operaciones concretas en los controladores especializados.

Por ejemplo, cuando el usuario pulsa el botón "Crear" dentro del módulo de citas, el Listener verifica el estado actual del panel panel_x, cambia la vista al formulario correspondiente mediante vent.cambiarCajaPrimario(), invoca al controladorCreacion.formularioCitas() para poblar los desplegables con datos actualizados, y establece el modo del panel a "Crear".

```
//Bloque else if creacion
} else if (e.getSource() == panel_citas.getBtn_cCita()) {
    // Boton confirmar en citas
    if (panel_citas.getModo().equals("Crear")) {
        controladorCreacion.crearCita();
    } else {
        controladorEdiciones.editarCita();
    }
    // Recargar tablas
    controladorTablas.cargarCitas();
    controladorTablas.citasRecientes();
    controladorTablas.cargarOcupacionTalleres();
    vent.cambiarCajaPrimario(panel_x);
}
```

De manera análoga, el botón "Login" desencadena una validación de credenciales con el modelo y, si es exitosa, ejecuta los métodos **iniciarMaestro()**, **iniciarOficial()** o **iniciarAprendiz()** según la categoría del empleado, adaptando dinámicamente la interfaz a sus permisos.

```
case "Login":
    //almaceno las credenciales
    String usuario = panel_login.getTextField_usuario().getText();
    String contraseña = panel_login.getPasswordField_contrasena();

    //inicio de la sesion
    sesion = modelo.login(usuario, contraseña);

    if (sesion != null) {
        panel_login.getLblErrorInicio().setText("");
        String tipoCuenta = sesion.getCategoria();
        vent.cambiarCajaCuenta(panel_cuenta);
        panel_cuenta.mostrarHome();
        if (tipoCuenta.equals("Maestro")) {
            iniciarMaestro();
        } else if (tipoCuenta.equals("Oficial")) {
            iniciarOficial();
        } else if (tipoCuenta.equals("Aprendiz")){
            iniciarAprendiz();
        }
        // barrido de las credenciales para el logout
        panel_login.getPass().setText("");
        panel_login.getTextField_usuario().setText("");
    } else {
        //mensaje de error y limpiado de los textos
        panel_login.getLblErrorInicio().setText("Usuario o contraseña invalidos, Acceso denegado");
        panel_login.getTextField_usuario().setText("");
        panel_login.getPass().setText("");
        panel_login.getTextField_usuario().requestFocus();
    }
    break;
```

La comunicación entre el Listener y el resto de los controladores se realiza mediante el intercambio de objetos y la actualización de las tablas a través de métodos como **controladorTablas.cargarCitas()** o **controladorTablas.citasRecientes()**.

Especialmente relevante es el uso del panel panel_x, que actúa como un contenedor multipropósito cuyo String estado determina en qué módulo se encuentra el usuario, permitiendo al Listener saber si debe operar sobre citas, clientes, empleados o talleres.


```

else if (e.getSource() == panel_talleres.getBtn_homeTaller()) {
    vent.cambiarCajaPrimario(panel_x);
    if (panel_x.getEstado().equals("talleres")) {
        controladorTablas.cargarTalleres();
    }
}

} else if (e.getSource() == panel_clientes.getBtn_homeClientes()) {
    vent.cambiarCajaPrimario(panel_x);
    if (panel_x.getEstado().equals("clientes")) {
        controladorTablas.cargarClientes();
    }
}

} else if (e.getSource() == panel_empleados.getBtn_homeEmpleado()) {
    vent.cambiarCajaPrimario(panel_x);
    if (panel_x.getEstado().equals("empleados")) {
        controladorTablas.cargarEmpleados();
    }
}

} else if (e.getSource() == panel_citas.getBtn_homeCitas()) {
    vent.cambiarCajaPrimario(panel_x);
    if (panel_x.getEstado().equals("citas")) {
        controladorTablas.cargarCitas();
    }
}
    
```

Por otra parte, el flujo completo de una operación de modificación ilustra perfectamente nuestra arquitectura; el usuario selecciona una fila en la tabla, pulsa "Modificar", el Listener obtiene la fila seleccionada mediante el cotrolador de ediciones y el uso del método **getFilaSeleccionada()**, carga los datos en el formulario a través del método **cargarCitaParaEditar()**, **cargarClienteParaEditar()**, etc.), cambia el modo del panel a "Modificar" y, cuando el usuario confirma los cambios, el botón "Confirmar" cuyo origen es el propio panel del formulario ejecuta el bloque correspondiente que llama a **controladorEdiciones.editarCita()**. De esta manera, el Listener actúa como un verdadero comandante general que no ensucia su código con lógica de negocio detallada, sino que se limita a orquestrar y dirigir el tráfico de eventos hacia los controladores especializados, resultando en una aplicación cohesiva, desacoplada y preparada para futuras ampliaciones.

```

case "Modificar":
    // Solo edición
    if (panel_x.getEstado().equals("citas")) {
        Object[] fila = controladorEdiciones.getFilaSeleccionada();
        if (fila != null) {
            vent.cambiarCajaPrimario(panel_citas);
            controladorCreacion.formularioCitas(); // Para llenar combos
            controladorEdiciones.cargarCitaParaEditar(fila);
            panel_citas.setModo("Modificar");
        } else {
            //prints para actualizacion de dialogs
            System.out.println("Seleccione una fila para modificar");
        }
    }
    
```

Para facilitar las pruebas de funcionamiento durante el desarrollo, se insertaron mensajes de depuración (**System.out.println()**) en puntos estratégicos de cada método, tanto en el Listener como en el resto de controladores y en el modelo. Estos prints permitieron al equipo seguir el rastro de ejecución paso a paso, identificando rápidamente si un método se invocaba correctamente, si los valores obtenidos de la base de datos eran los esperados o si alguna condición derivaba en un flujo alternativo no deseado.

```
===
Boton presionado: Confirmar
Cliente modificado exitosamente. ID: 8
Traje modificado exitosamente
Cliente y traje modificados exitosamente
Cliente modificado correctamente
Limpiando formulario de cliente
Cambiado el estado a clientes
===
Boton presionado: Talleres
Cambiado el estado a talleres
===
Boton presionado: Eliminar
Eliminando taller: Tokio (ID: 7)
Taller eliminado exitosamente. ID: 7
Cambiado el estado a talleres
```

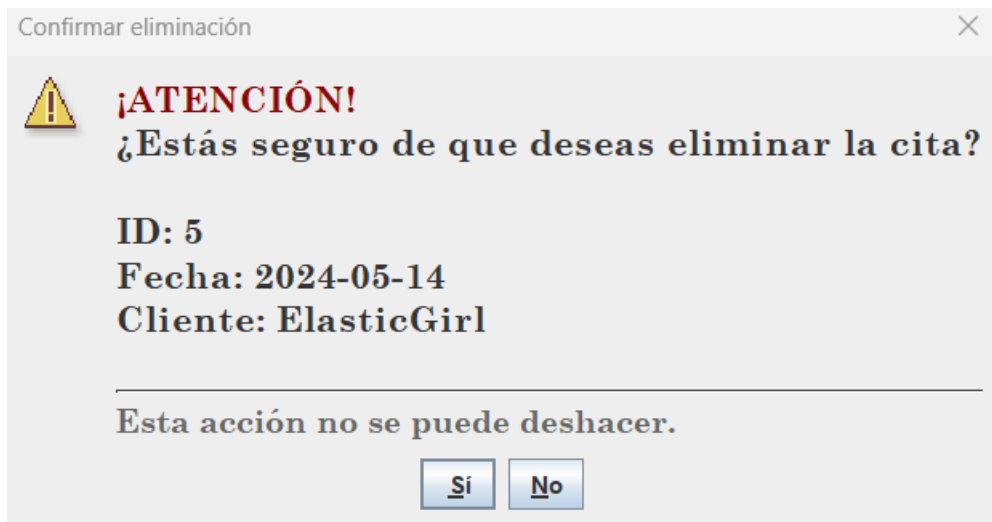
Por ejemplo, en el método login() del modelo se incluyó un mensaje que confirmaba su invocación, mientras que en crearCita() se volcaban avisos específicos como "Cliente no encontrado" o "Cita creada exitosamente". Esta estrategia, aunque sencilla, resultó especialmente valiosa y aceleró notablemente la detección y corrección de errores tanto en la lógica de negocio como en las interacciones entre paquetes.

```
System.out.println("No hay trajes disponibles para: " + seleccion);
System.out.println("ERROR: Campos vacíos en el formulario");
System.out.println("Fallo al crear el cliente");
System.out.println("Eliminación cancelada por el usuario");
JOptionPane.showMessageDialog(null, "Cita eliminada correctamente", "Éxito", JOptionPane.INFORMATION_MESSAGE);
```

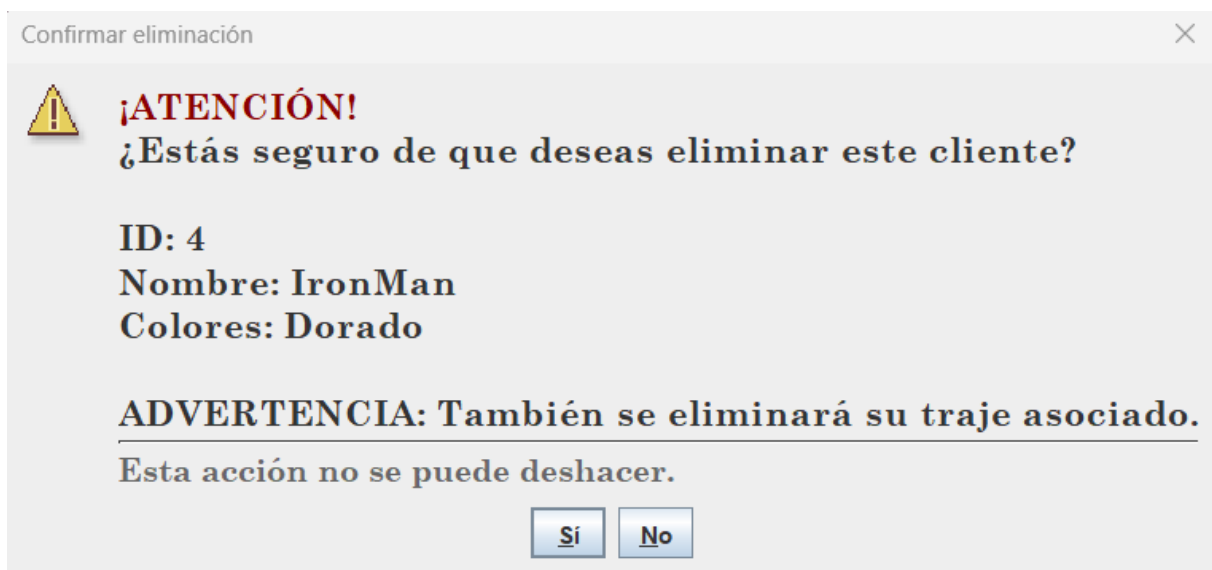
Estos mensajes de depuración se complementaron con una interfaz de diálogos visuales pensada para el usuario final.

En el controlador de eliminaciones, por ejemplo, se implementó el método mostrarConfirmacion(), que genera ventanas emergentes estilo JOptionPane con formato HTML personalizado.

Este diálogo muestra mensajes de advertencia en color burdeos, utiliza la tipografía corporativa Century Schoolbook e incluye un separador y un texto informativo ("Esta acción no se puede deshacer") para alertar al empleado sobre la irreversibilidad de la operación.



Dependiendo del tipo de entidad a eliminar, cita, cliente, empleado o taller el mensaje se construye dinámicamente mostrando los datos clave del registro seleccionado, como el ID, la fecha y el cliente en el caso de una cita, o el nombre y la categoría en el caso de un empleado. Además, si la operación tiene consecuencias en cadena como eliminar un cliente que conlleva el borrado automático de su traje asociado el mensaje lo advierte explícitamente.



Tras la confirmación del usuario, se muestra otro diálogo de éxito o error según el resultado de la operación. De esta manera, la aplicación no solo informa al desarrollador mediante mensajes en consola, sino que también guía y protege al empleado en sus acciones críticas, mejorando tanto la experiencia de usuario como la seguridad operativa del sistema.

Completada la implementación de todas las clases que conforman la Vista, Modelo y Controlador, el equipo dispuso de un esqueleto funcional completo de la aplicación. Cada clase, desde el Panel_login hasta Control_eliminacion, pasando por Acceso_BD y las clases de consulta cumplía con la responsabilidad que se le había asignado inicialmente. No obstante, tener todas las piezas ensambladas no significaba que el programa estuviese listo para su entrega. Era necesario someter el conjunto a un proceso continuo de refinamiento, depuración de errores, ajustes de usabilidad y optimización del rendimiento. Este siguiente paso resultó tan crucial como la propia codificación, pues permitió pulir los fallos propios de todo desarrollo software y acercar la aplicación al nivel de calidad y fiabilidad que se exige para un proyecto de esta magnitud.

La refinación a este esqueleto funcional comenzó por la implementación de arreglos en la estructura visual del apartado más importante, la creación de citas, donde entre otras cosas se añadieron funcionalidades extra que mejoraban el uso y experiencia de usuario en el programa.

Crear una cita

Encargado: Edna Fecha: ...

Asistente #1: Sin asistente Hora:

Asistente #2: Sin asistente Duración: 1

Cliente: Spiderman + Nuevo Cliente

Traje: Hombre araña + Nuevo Traje

Taller: Paris

Confirmar

Uno de los primeros cambios significativos en el proceso de refinamiento afectó a la selección de fecha y hora. Inicialmente, estos campos eran simples JTextField donde el usuario debía introducir manualmente los datos, lo que generaba frecuentes errores de formato y complicaba la inserción en la base de datos, además de ofrecer una experiencia de usuario pobre. Para solucionarlo, se integró la librería externa LGoodDatePicker (cuyo funcionamiento se detalla en el Anexo III). Gracias a ella, se sustituyeron los campos de texto por un DatePicker con calendario visual y un TimePicker con menú desplegable, logrando un aspecto más refinado y profesional. Esta mejora no solo facilitó la tarea al empleado, que ahora selecciona fecha y hora con un simple clic, sino que también permitió aplicar una validación esencial: no se puede elegir una fecha anterior al día actual, alineándose con la lógica evidente de cualquier sistema de citas. Desde el punto de vista del desarrollo, la librería devuelve los valores en el formato estándar yyyy-MM-dd y HH:mm, compatibles directamente con el tipo DATE y TIME de MySQL, eliminando conversiones manuales y reduciendo errores en la capa de persistencia.

Fecha:

Hora:

Duración:

Mayo 2026

lun	mar	mié	jue	vie	sáb	dom
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

Hoy: 3 may 2026 Borrar

Continuando con las mejoras visuales, el TimePicker se configuró para mostrar un desplegable con las horas permitidas según el horario laboral de los talleres, que opera de 9:00 a 20:00 horas. Esta selección por intervalos de treinta minutos no solo facilitó la inserción en la base de datos al garantizar que el valor introducido respeta el formato TIME de MySQL, sino que también sentó la base para implementar reglas de negocio fundamentales en la gestión de citas.

Por un lado, disponer de horas discretas y controladas permitió validar que dos citas no pudieran solaparse en el mismo taller, comprobando si los intervalos horarios se cruzaban. Por otro lado, la precisión de los horarios hizo posible detectar cuándo una cita de un héroe y otra de un villano quedaban demasiado cerca (con menos de una hora de diferencia), evitando así encuentros no deseados en las instalaciones.

Otro de los añadidos fundamentales en el apartado de citas fue la implementación de botones para acciones rápidas que centralizan las operaciones más frecuentes en el programa; los botones "**Nuevo Cliente**" y "**Nuevo Traje**". Antes de su incorporación, si durante la creación de una cita el empleado detectaba que el cliente no estaba registrado o que el traje que necesitaba el cliente aún no existía en el sistema, debía cancelar el formulario, dirigirse al módulo correspondiente, realizar el alta y luego volver a empezar. Esta navegación resultaba tediosa y rompía el flujo natural de trabajo.

Con los nuevos botones, situados estratégicamente junto a los desplegados de cliente y traje, se abre una ventana emergente que permite dar de alta un nuevo cliente o traje sin abandonar el formulario de citas. Una vez completado el registro, los desplegados se recargan automáticamente y el nuevo elemento aparece seleccionado por defecto, agilizando así la continuidad del proceso.

Técnicamente, la implementación requirió coordinar la vista (Panel_citas) con los controladores de creación (Control_creacion), reutilizando los mismos métodos que se emplean en los formularios específicos de clientes y trajes, pero adaptados a un diálogo más compacto.

Esta adaptación se dio mediante métodos específicos en la clase Control_creacion que manejan internamente la lógica de creación por ventana, en estos métodos se invoca un JOptionPane que contiene los elementos necesarios para generar un nuevo cliente o traje

```

/*
 * Método para ventana emergente al crear un nuevo traje
 * dentro de la creación de citas
 */
@SuppressWarnings({ "rawtypes", "unchecked" })
public void crearTrajeVentana() {

    JComboBox cliente = new JComboBox();
    JTextField nombre = new JTextField();

    // Llenado combo de clientes
    ArrayList<Cliente> clientes = consultas_cliente.mostrarClientes();
    if (clientes != null) {
        for (Cliente n : clientes) {
            cliente.addItem(n.getNombre());
        }
    }

    Object[] diseñoFormulario = {
        "Cliente:", cliente,
        "Nombre:", nombre
    };

    Object[] botones = {"Cancelar", "Aceptar"}; // Cancelar a la izquierda
    int botonPulsado = JOptionPane.showOptionDialog(
        null,
        diseñoFormulario,
        "Datos del nuevo traje",
        JOptionPane.OK_CANCEL_OPTION,
        JOptionPane.INFORMATION_MESSAGE,
        null,
        botones,
        botones[1] // botón por defecto al pulsar Enter
    );
}

```

Además, se añadió la lógica de refresco de los JComboBox correspondientes invocando a los métodos cargarClientes() y cargarTrajesPorCliente() una vez finalizada la inserción. De esta manera, la aplicación mantiene la coherencia de datos y ofrece una experiencia fluida y profesional, alineada con la filosofía de eliminar fricciones innecesarias en el día a día del taller.

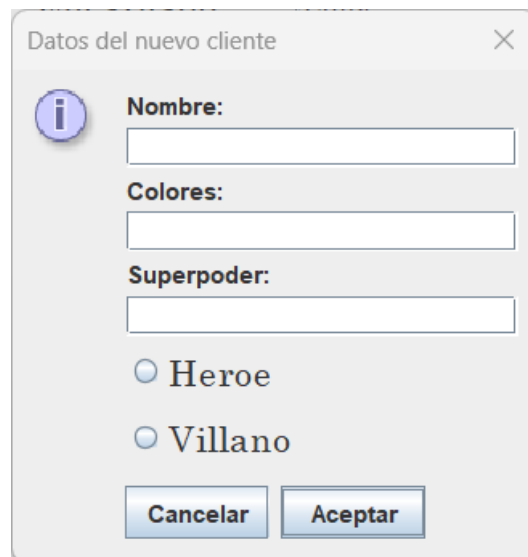
```

if (creacion) {
    //mensaje de confirmacion
    JOptionPane.showMessageDialog(null, "Traje " + nombreGuardado + " añadido exitosamente", "Exito",
    JOptionPane.INFORMATION_MESSAGE);
}

// Recarga los trajes del cliente para que aparezca el traje, y lo selecciona
cargarTrajesPorCliente();
panel_cita.getCbTrajes().setSelectedItem(nombreGuardado);

} else {
    System.out.println("El usuario canceló la creación del traje.");
}
    
```

La implementación de estas ventanas emergentes añadió una mayor capa de profundidad técnica al panel de citas y, a su vez, nos dio al equipo una sólida idea de cómo manejar la mejora del programa en otros aspectos.



El aprender a lanzar diálogos emergentes que mantienen el contexto de la ventana principal, reutilizar la lógica de creación existente y refrescar dinámicamente los componentes afectados se convirtió en un patrón que aplicamos posteriormente en otras áreas. Por ejemplo, el botón "Editar traje" dentro del formulario de clientes sigue el mismo principio; abre una ventana emergente que permite modificar el nombre o el estado de un traje sin salir del panel de cliente.

Datos del Traje

Nombre:

Editar traje

Esta funcionalidad resultó un poco más especial de integrar, ya que la opción de editar el traje solo debe aparecer cuando se está modificando un cliente existente, no mientras se crea uno nuevo. Para lograrlo, hubo que jugar con el método `.setVisible()` de los componentes de Swing dentro de un método exclusivo para controlar esta visibilidad condicional.

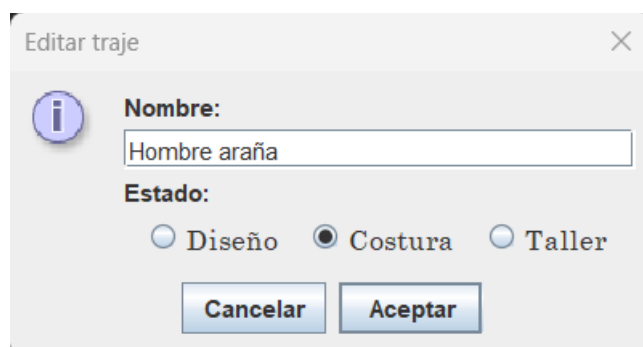
```

/**
 * Metodo para cambiar el aspecto del panel de cliente segun el modo elegido
 *
 * @param panel_clientes panel a modificar
 * @param modo String con el modo, puede ser crear / modificar
 */
public static void estadoEdicionCliente(Panel_clientes panel_clientes, String modo) {
    if (modo.equals("crear")) {
        //ocultar apartados de edicion
        panel_clientes.getBtnEditarTraje().setVisible(false);
        panel_clientes.getComboTrajes().setVisible(false);

        // mostrar apartados de creacion
        panel_clientes.getTfNombreT().setVisible(true);
        panel_clientes.getLblEstado().setVisible(true);
        panel_clientes.getRdbtnDiseno().setVisible(true);
        panel_clientes.getRdbtnCostura().setVisible(true);
        panel_clientes.getRdbtnTaller().setVisible(true);
    } else if (modo.equals("modificar")) {
        //mostrar apartados de edicion
        panel_clientes.getBtnEditarTraje().setVisible(true);
        panel_clientes.getComboTrajes().setVisible(true);

        // the samee
        panel_clientes.getTfNombreT().setVisible(false);
        panel_clientes.getLblEstado().setVisible(false);
        panel_clientes.getRdbtnDiseno().setVisible(false);
        panel_clientes.getRdbtnCostura().setVisible(false);
        panel_clientes.getRdbtnTaller().setVisible(false);
    }
}
    
```

Se creó el método `.estadoEdicionCliente()` en la clase de Utilidades, que recibe un String de parámetro indicando si el panel está en modo "crear" o "modificar". En el primer caso, el botón "Editar traje" y el desplegable de trajes se ocultan, ya que aún no hay un traje asociado al cliente; en el segundo, se hacen visibles para permitir modificar el nombre o el estado del traje seleccionado. Este enfoque no solo mantiene limpia la interfaz, evitando opciones que no corresponden al contexto, sino que también previene errores de lógica, como intentar editar un traje que todavía no existe.



Con la programación de todos los métodos para las ventanas emergentes llegó el momento de integrarlos en el flujo general de eventos de la aplicación. Para ello, se añadieron las correspondientes entradas en el switch de botones de la clase Listener central, el gran comandante que orquesta todas las acciones del usuario.

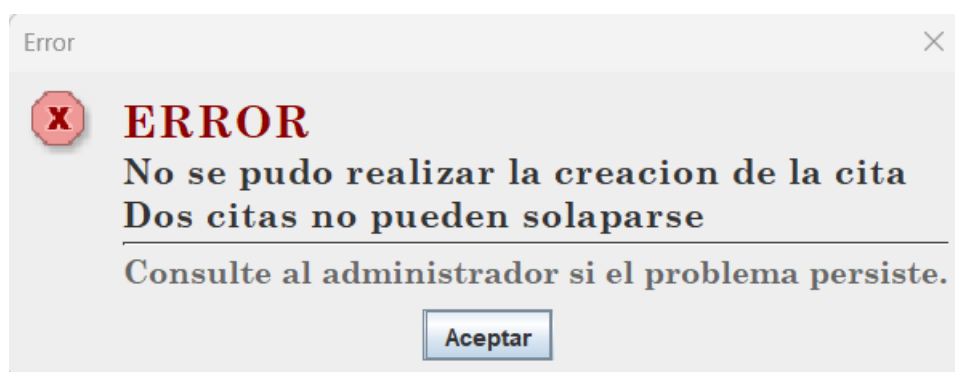
- "Nuevo Cliente"
- "Nuevo Traje"
- "Editar traje"
- Cada nuevo botón

Cada nuevo botón recibió su propio **case** dentro del bloque de eventos por boton. De esta forma, cuando el empleado pulsaba cualquiera de estos botones, el Listener capturaba la acción y delegaba inmediatamente en el método correspondiente del controlador de creación o de ediciones, sin necesidad de modificar la lógica existente



Como último refinamiento necesario para el programa, dentro del apartado de citas, se contempló y abordó una de las piezas de lógica de negocio más importantes; la validación de solapamiento de citas y el control de cruce entre héroes y villanos.

Para la primera, se implementó un algoritmo que, dadas una fecha, un taller, una hora de inicio y una duración, compara la nueva cita con todas las existentes en la base de datos.



Se calculan los extremos temporales y se verifica si los intervalos se solapan mediante cuatro condiciones:

- si el comienzo de la cita creada esta entre la cita comprobada
- si el final de la cita creada esta entre la cita comprobada
- si el comienzo de la cita comprobada esta entre la cita creada
- si el final de la cita comprobada esta entre la cita creada

Esta lógica, aparentemente sencilla, requirió múltiples pruebas para manejar correctamente los casos y la conversión precisa de minutos a fracciones de hora.

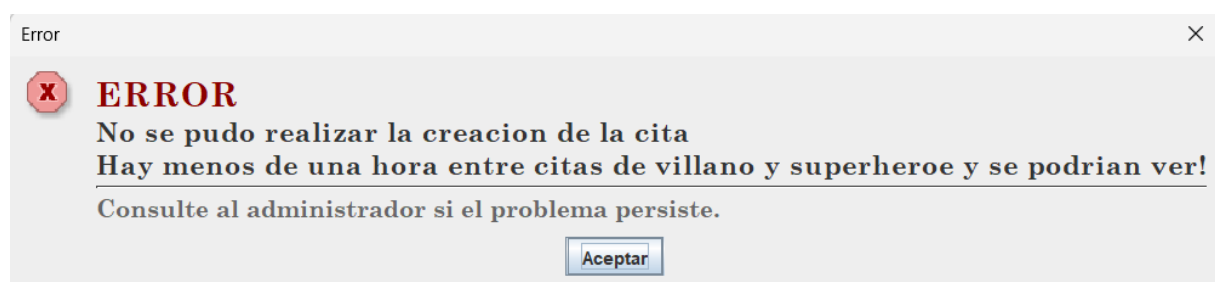
```
// Obtener duracion de la cita
int duracion = (int) panel_cita.getSpDuracion().getValue();

// Validar que no solape con ninguna otra cita, y que no se vean heroes y villanos
for (Cita c : consultas_cita.mostradoCitas()) {
    // Si esta en el mismo día y mismo taller
    if (c.getFecha().equals(fecha) && c.getId_taller() == ids.obtenerIdTaller(taller)) {
        // convierte la hora en un timestamp numerico del día
        String[] tiempos = hora.split(":");
        // timestamps cita creada
        double t1 = Double.parseDouble(tiempos[0]) + Double.parseDouble(tiempos[1])/60;
        double t1f = t1 + duracion;

        // timestamps cita comprobada
        double t2 = c.getHoraInt() + ((double) c.getMinutoInt())/60;
        double t2f = t2 + c.getDuracionInt();

        // Comprobacion de solapacion
        boolean solapan = false;
        // si el comienzo de la cita creada esta entre la cita comprobada...
        if (t1 > t2 && t1 < t2f) {
            solapan = true;
        } // si el final de la cita creada esta entre la cita comprobada...
        } else if (t1f > t2 && t1f < t2f) {
            solapan = true;
        } // si el comienzo de la cita comprobada esta entre la cita creada...
        } else if (t2 > t1 && t2 < t1f) {
            solapan = true;
        } // si el final de la cita comprobada esta entre la cita creada...
        } else if (t2f > t1 && t2f < t1f) {
            solapan = true;
        }
    }
}
```

Paralelamente, se introdujo una regla específica para el taller de Edna; una cita de un héroe no puede programarse con menos de una hora de diferencia respecto a una cita de un villano en el mismo taller, para evitar encuentros no deseados.



Para ello, se obtienen las alineaciones de ambos clientes mediante una consulta a la base de datos en la clase de ConsultasCliente; este método, llamado `alineacionCliente()` recibe como parámetro el id de un cliente para devolver un objeto de tipo `String` que contiene la alineación relacionada a este id en la base de datos.

```
/**
 * Metodo que devuelve la alineacion (Heroe / Villano)
 * de un cliente en la base de datos
 *
 * @param id identificador del cliente
 * @return String con su alineacion
 */
public String alineacionCliente(int id) {

    String query = "SELECT alineacion FROM Cliente WHERE id_cliente = ?";

    try (PreparedStatement stmt = instance.prepareStatement(query)) {
        stmt.setInt(1, id);

        try (ResultSet resultado = stmt.executeQuery()) {
            if (resultado.next()) {
                return resultado.getString("alineacion");
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return null;
}
```

Si las alineaciones son opuestas, se calcula la distancia entre el final de una cita y el comienzo de la otra. Si esta distancia es inferior a una hora, el sistema bloquea la creación o modificación y muestra un mensaje de error personalizado.

```
// Comprobación de héroes y villanos
String tcCreado = consultas_cliente.alineacionCliente(c.getId_cliente());
String tcComprobado = consultas_cliente.alineacionCliente(ids.obtenerIdCliente(cliente));

// Si los dos son diferentes (héroe vs villano)
if (tcCreado != null && tcComprobado != null && !tcCreado.equals(tcComprobado)) {
    boolean pelea = false;
    // Si el final de la cita está a menos de una hora del comienzo de la otra
    if (Math.abs(t1f - t2) < 1) {
        pelea = true;
    }
    // Si el comienzo de la cita está a menos de una hora del final de la otra
    } else if (Math.abs(t1 - t2f) < 1) {
        pelea = true;
    }
    }

    if (pelea) {
        String error = "No se pudo realizar la modificacion de la cita\n" +
            "Hay menos de una hora entre citas de villano y superheroe";
        confirm.mostrarError("Error", error);
        System.out.println("ERROR: Citas de villano y superheroe demasiado cercanas");
        return false;
    }
}
```

Ambas validaciones se integraron tanto en el método crearCita() como en editarCita(), asegurando que las reglas se apliquen siempre, independientemente de la operación.

Para dar cierre a este apartado de desarrollo, queremos reflexionar como a pesar de su extensión, no hace completa justicia a todo el tiempo que realmente tomó la programación de la aplicación. Detrás de cada párrafo hay horas de depuración, discusiones de equipo, reescritura de código y pruebas fallidas. La implementación de la lógica de solapamiento, por ejemplo, requirió tres enfoques diferentes hasta dar con el que manejaba correctamente los casos limite; la integración de las ventanas emergentes obligó a rediseñar parte del flujo de eventos; y la coordinación entre los distintos paneles y el Listener necesitó múltiples iteraciones para lograr un funcionamiento fluido.

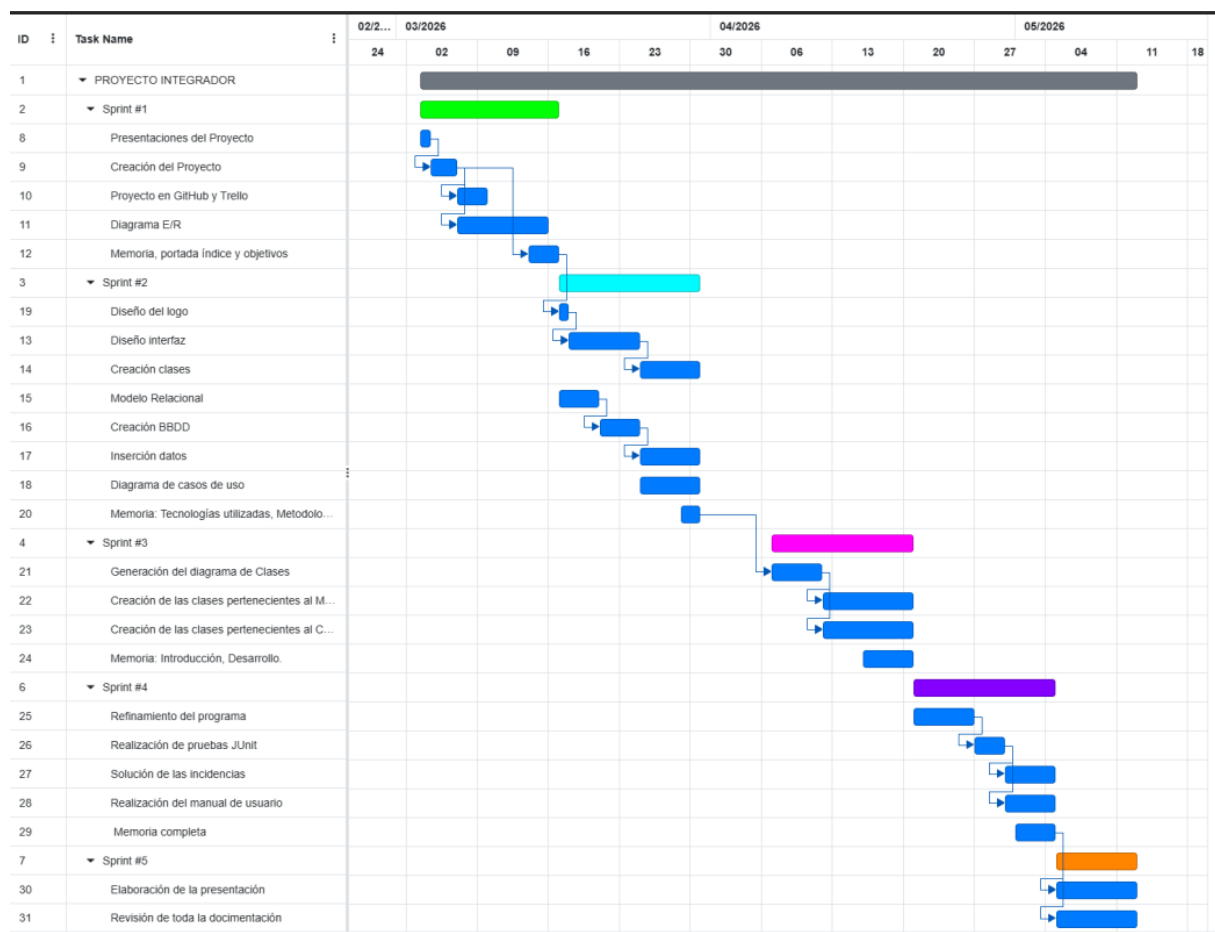
Lo que aquí se resume en unas líneas fue un proceso continuo de aprendizaje, paciencia y perseverancia. Cada error nos enseñó algo nuevo, y cada acierto nos impulsó a seguir mejorando. Por ello, este documento no pretende ser una crónica exhaustiva de cada obstáculo, sino un reflejo fiel del esfuerzo colectivo y de la evolución de un grupo de estudiantes que, con más ilusión que experiencia, logró convertir una idea en un producto funcional.

5. Metodología

Como metodología para la realización de este proyecto se adoptó el marco de trabajo ágil SCRUM, cuya gestión de proyectos se da mediante cortos ciclos planificados llamados sprints; para este proyecto en específico se definieron un total de 4 diferentes sprints de 2 semanas de duración, cada uno abarcando la totalidad del desarrollo de la aplicación gestora de citas.

- Primer Sprint: 3 de marzo de 2026 – 16 de marzo de 2026
- Segundo Sprint: 17 de marzo de 2026 – 30 de marzo de 2026
- Tercer Sprint: 7 de abril de 2026 – 20 de abril de 2026
- Cuarto Sprint: 21 de abril de 2026 – 4 de mayo de 2026

El primer paso al adoptar la metodología SCRUM fue el definir roles, tareas y plazos para la realización y entrega de cada una de las tareas generadas a lo largo del proyecto. Para modelar esta organización representamos los plazos de cada tarea y cada sprint con la ayuda del diagrama de Gantt; en este diagrama cada tarea es representada con una grafica en el eje vertical, donde se determina su tiempo de duración en un cronograma hacia el eje horizontal.



En el diagrama se represento cada sprint con un color diferente como ayuda visual a la hora de saber que tareas (graficas en color azul) se desprenden directamente de ese sprint.

Primer Sprint: 3 de marzo de 2026 – 16 de marzo de 2026

En este periodo se realizó la planificación inicial del proyecto, donde se evaluaron las especificaciones de este y la planificación detallada de los componentes de la aplicación a diseñar, las tareas realizadas en este sprint fueron:

- Creación repositorio en GitHub del proyecto.
- Creación de tablero de seguimiento de tareas en Trello.
- Planificación detallada de los requisitos de la aplicación.
- Creación diagrama Entidad-Relación.
- Inicio de redacción en memoria en los apartados de portada, índice y objetivos.

Segundo Sprint: 17 de marzo de 2026 – 30 de marzo de 2026

Con las bases del proyecto ya sentadas en el primer sprint, se dio paso a la realización de las tareas que sentaron la base del desarrollo practico de la aplicación gestora de citas, estas tareas fueron:

- Creación del modelo Relacional.
- Creación de la base de datos de la aplicación.
- Esquemmatización del modelo de casos de uso
- Diseño de la interfaz de las ventanas de la aplicación.
- Diseño del logo y branding empresarial.
- Redacción de apartados en la memoria: Tecnologías utilizadas, Anexo 1 y metodología.

Tercer Sprint: 7 abril de 2026 – 20 de abril de 2026

El antecedente del segundo abrió paso a las tareas mas cruciales del desarrollo del proyecto:

- Generación del diagrama de clases: Diagrama UML de clases.
- Creación de las clases pertenecientes al Modelo
- Creación de las clases pertenecientes al Controlador
- Configuración del acceso a la base de datos y del manejo de la aplicación.
- Redacción de los apartados de la memoria Introducción y Desarrollo.

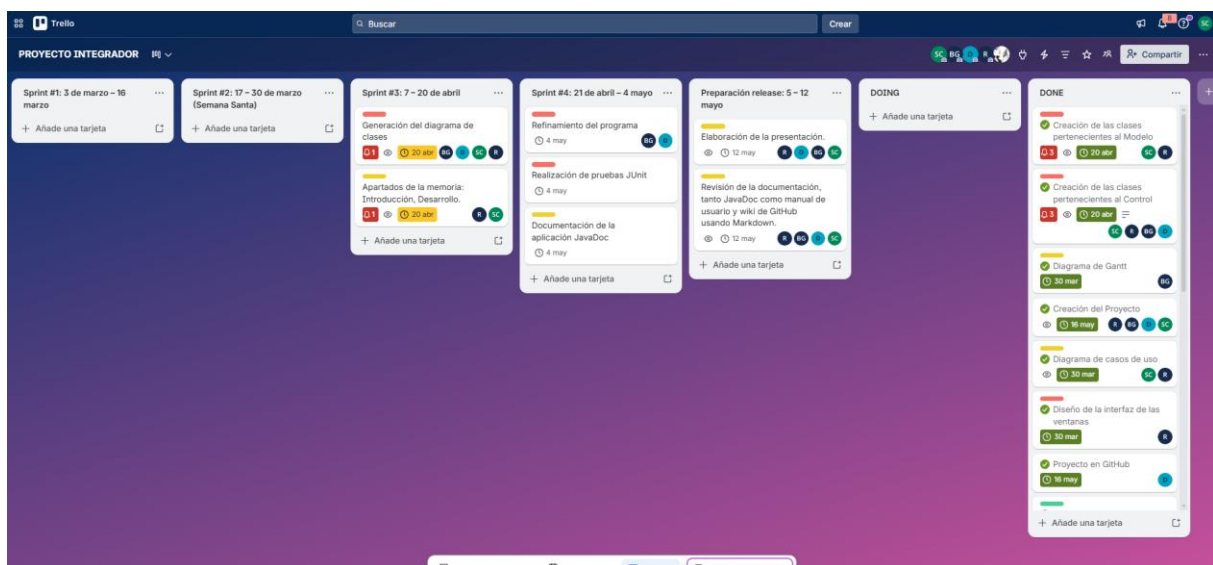
Cuarto Sprint: 21 de abril de 2026 – 4 de mayo de 2026:

Con el sprint #3 superado ya existe una aplicación funcional en la que se pueden realizar toda la gama de acciones CRUD, pero esta aun se encuentra en una etapa muy cruda, donde las cosas funcionan, pero están susceptibles a muchos fallos; por esta razón el puto central de este sprint es el mejorar lo ya construido mediante las siguientes tareas:

- Refinamiento del programa
- Documentación de la aplicación con javaDoc
- Solución de errores en el código, mejora en interacciones del usuario y solución a posibles fallos durante la ejecución del programa
- Realización del manual de usuario en la wiki de GitHub

Como parte del marco de trabajo de la metodología SCRUM, se utilizó la herramienta “Trello” para la planificación y seguimiento de las tareas. Se creó un tablero colaborativo estructurado en las columnas: Doing, Done y una por cada sprint a realizar. Cada tarea se representó mediante una tarjeta con su descripción y área a la que pertenece la tarea.

La distribución de tareas se realizó al inicio de cada sprint. Se analizó y asignó cada tarea a uno o varios miembros según sus habilidades, tiempo disponible y nivel de complejidad de la funcionalidad a desarrollar.



De esta manera, Trello permitió al equipo mantener una visión transparente del progreso del proyecto dentro del marco de trabajo ágil SCRUM adoptado.

6. Resultados y conclusiones

Al iniciar este proyecto el equipo se enfrentó a un escenario que a primera vista resultaba abrumador. La magnitud de la aplicación requerida, un sistema completo de gestión de citas, clientes, trajes, empleados y talleres parecía un desafío gigantesco para un grupo de estudiantes con poca experiencia en el manejo de proyectos a esta escala.

Sin embargo, a lo largo de las semanas, el sentimiento inicial de incertidumbre se transformó gradualmente en motivación y orgullo. La adopción de una metodología ágil como SCRUM, permitió al equipo afrontar el proyecto por partes, descomponiendo un problema complejo en pequeñas tareas abordables. Ver cómo semana a semana, la aplicación adquiría forma y funcionalidad; desde las primeras consultas a la base de datos hasta la interfaz gráfica completamente operativa se convirtió en el principal combustible para mantener el esfuerzo y la dedicación hasta el último momento del desarrollo.

Uno de los aprendizajes más valiosos que extraemos de esta experiencia es la constatación de que las fases previas a la programación; el diseño del modelo Entidad-Relación, la elaboración del diagrama de clases UML, la planificación de sprints y el prototipado de wireframes resultaron, en retrospectiva, incluso más determinantes para el éxito del proyecto que la propia programación y escritura de código.

Comprender la estructura de la base de datos antes de escribir una sola línea de MySQL o Java, definir las relaciones entre entidades y anticipar los flujos de navegación del usuario nos ahorró innumerables horas de depuración y reescritura de código. El modelado previo actuó como un mapa que guió cada decisión técnica, y cada vez que nos desviábamos de ese mapa, el proyecto nos lo recordaba con errores y contradicciones. Aprendimos así que picar código es solo la punta del iceberg del desarrollo de software, y que la verdadera ingeniería reside en la planificación, el análisis y el diseño.

En el plano personal y grupal, el proyecto nos ha permitido aplicar de forma práctica todos los conocimientos adquiridos a lo largo del curso:

- programación orientada a objetos en Java
- bases de datos relacionales con MySQL
- diseño de interfaces gráficas con Swing
- control de versiones con Git y GitHub
- trabajo colaborativo bajo una metodología ágil

Pero más allá de lo técnico, la gestión de un equipo de cuatro personas con diferentes ritmos, fortalezas y horarios nos ha enseñado lecciones inolvidables sobre comunicación, distribución de tareas, resolución de conflictos y, sobre todo, sobre la importancia de la confianza mutua. Los imprevistos que surgieron, desde dificultades técnicas hasta solapamientos de calendarios fueron inevitables, pero el grupo supo sortearlos gracias a una comunicación fluida y a la voluntad colectiva de sacar el proyecto adelante.

El resultado final es una aplicación funcional, estable y alineada con los requisitos establecidos en el planteamiento inicial. La interfaz, diseñada a partir de los wireframes creados, resulta intuitiva y agradable para el usuario, adaptando sus opciones en función de la categoría del empleado autenticado. La capa de seguridad en la base de datos garantiza la integridad de los datos mediante claves foráneas bien definidas y validaciones tanto a nivel de interfaz como de servidor. La arquitectura Modelo-Vista-Controlador se ha utilizado provechosamente, lo que facilitará futuras ampliaciones y el mantenimiento del código.

En conclusión, aunque el camino no fue exento de obstáculos, el equipo se siente profundamente orgulloso del trabajo realizado. Nos hemos demostrado que, con planificación, esfuerzo y colaboración, es posible transformar una idea inicialmente intimidante en un producto software tangible y de calidad. Este proyecto no solo ha consolidado nuestras competencias técnicas, sino que nos ha dejado una lección fundamental para nuestra futura carrera profesional

“el desarrollo de software es, ante todo, un trabajo en equipo donde la preparación previa y la capacidad de adaptarse a los cambios valen tanto como la habilidad para escribir código”

7. Trabajos futuros

A pesar de que la aplicación desarrollada cumple con todos los requisitos funcionales establecidos en el planteamiento del proyecto inicial y ofrece una experiencia de usuario completa para los empleados del taller de Edna, el equipo es consciente de que el software podría ampliarse y mejorarse significativamente. Durante el proceso de desarrollo surgieron numerosas ideas que, por limitaciones de tiempo, no pudieron ser implementadas en esta versión. A continuación, se presentan las principales líneas de mejora que como desarrolladores nos hubiera gustado implementar en el proyecto.

Versión cliente de la aplicación

La funcionalidad que el equipo tenía más ilusión por desarrollar es, sin duda, una versión cliente de la aplicación. En la implementación actual, el software está orientado exclusivamente a los empleados del taller, que son quienes gestionan las citas, los trajes y los talleres. Sin embargo, una de las carencias evidentes es la ausencia de un canal de comunicación directo entre el taller y los propios superhéroes y supervillanos que confían en Edna para el diseño y confección de sus trajes.

La versión cliente permitiría a los usuarios acceder al sistema con sus propias credenciales, visualizar el estado de sus trajes, consultar el historial de citas pasadas y, lo más importante, gestionar sus propias citas sin necesidad de empleados que actúen como intermediarios.

Esta funcionalidad no solo supondría una mejora en el valor del proyecto, sino que también nos ayudaría como futuros desarrolladores a comprender como un software puede ser escalado y versionado, el entender como de un todo principal se da paso a ideas y aplicaciones mas especializadas que dan mayor vida a un proyecto y demuestran el valor que tenemos como programadores.

Sistema de calificación por estrellas y ranking de empleados

Estrechamente ligado a la versión cliente, el equipo ideó un sistema de calificación por estrellas que permitiría a los clientes valorar la calidad del servicio recibido al finalizar cada cita. Dichas calificaciones, en una escala de una a cinco estrellas, se acumularían en el perfil de cada empleado, generando un ranking interno de trabajadores que reflejaría su desempeño, profesionalidad y trato con los clientes.

Esta mecánica, inspirada en plataformas como Uber o Airbnb, introduciría un componente de gamificación en el taller que podría tener múltiples beneficios en el proyecto. Los empleados mejor valorados podrían optar a encargos más exclusivos como el diseño de trajes para clientes VIP o la participación en proyectos especiales, lo que incentiva la calidad del servicio y la mejora continua. Además, los maestros contarían con una herramienta objetiva para la toma de decisiones sobre asignación de responsabilidades o detección de áreas de mejora en determinados empleados.

Calendario interactivo de citas

Otra de las mejoras que el equipo consideró como una idea para un futuro desarrollo, es la sustitución de la actual tabla de citas por un calendario interactivo de estilo mensual o semanal, similar a los que se encuentran en aplicaciones como Google Calendar o Outlook. Este componente visual permitiría a los empleados del taller visualizar de un solo vistazo la ocupación de cada taller en cada franja horaria, arrastrar y soltar citas para reprogramarlas, y detectar de forma intuitiva posibles conflictos de solapamiento o de cercanía entre citas de héroes y villanos.

La implementación de un calendario interactivo requeriría un esfuerzo de desarrollo adicional, tanto en la capa de Vista (con componentes Swing más avanzados o la integración de librerías específicas) como en la lógica de negocio para soportar la manipulación directa de citas mediante eventos de ratón y teclado. No obstante, el equipo considera que el salto en calidad del proyecto compensaría sobradamente el esfuerzo, especialmente en un entorno como un taller de diseño donde la agenda es un elemento central del día a día.

Anexos

Anexo I – Listado de requisitos de la aplicación

Para la óptima ejecución de la aplicación, se definieron los siguientes requisitos mínimos, los cuales son necesarios para garantizar su correcto funcionamiento.

Requisitos de Hardware

Se recomienda un equipo estándar con capacidad para ejecutar entornos gráficos basados en ventanas y soportar el funcionamiento simultáneo de los componentes de la aplicación. Las especificaciones mínimas establecidas fueron:

- Procesador: Intel Core i5 o AMD Ryzen 5, con una frecuencia de 2.0 GHz o superior.
- Memoria RAM: 4 GB como mínimo, recomendándose 8 GB para un rendimiento óptimo durante la ejecución concurrente de la aplicación y el servidor de base de datos.
- Almacenamiento: 5 GB de espacio libre disponible para la instalación de la aplicación, sus dependencias y los archivos generados durante su ejecución.
- Sistema operativo: Windows 10 o superior, macOS, o cualquier distribución Linux con soporte para entornos gráficos de escritorio.

Requisitos de Software

Se definieron los siguientes componentes de software como necesarios para el despliegue y funcionamiento de la aplicación:

- Entorno de ejecución Java: Dado que la aplicación fue desarrollada en Java, se estableció como requisito indispensable contar con el Java Runtime Environment (JRE) versión 21 o, en su defecto, el Java Development Kit (JDK) 21, el cual incluye el entorno de ejecución.
- Servidor de base de datos: La aplicación requiere conectarse a un sistema gestor de bases de datos relacional para la persistencia de la información relacionada con citas, clientes, talleres y empleados. Para ello, se utilizó MySQL Server, el cual debió estar instalado y en ejecución en el equipo local o en un servidor accesible desde la aplicación. Se requirió que el servicio este activo y que se cuente con las credenciales de acceso adecuadas para la conexión.

Anexo II – Guía de uso de la aplicación

Incluirse en este anexo capturas de la aplicación, a modo de manual de usuario, incluyendo tanto el acceso de usuario, como de admin.

Anexo III – Desarrollo corporativo

El inicio del desarrollo de nuestro branding empresarial se remonta a una lluvia de ideas inicial del nombre que representaría a nuestro producto; nuestra intención principal era transmitir elegancia y posicionamiento de marca “élite”. Paralelo a la búsqueda del nombre buscábamos una representación gráfica que fuese sencilla y memorable para los clientes sin contradecir a los valores de elegancia y prestigio que la marca quiere transmitir.



Después de evaluar la mayoría de las propuestas nos decantamos por el uso del nombre “AntiCape-Corp”; la elección de este nombre se basó en dos principios. En primer lugar, la influencia de la política interna de la clienta Edna Moda cuya mayor normativa de diseño es “¡Nada de capas!”. No quisimos que esto quedara como un simple eslogan, por lo que planteamos varias ideas que representaran esta icónica referencia a Edna y AntiCape cumplía con este rol. Como pauta final para la elección del nombre, pensamos en agregar una abreviatura que sumara peso empresarial y estatus comercial a la marca, la alternativa que más concordaba con este planteamiento es la abreviatura a “corporation”, es decir, “Corp”.

En referencia a la elección del diseño del logo, quisimos que fuese sencillo pero elegante a su vez por lo que nos inspiramos en la tipografía “Ribon 131” la cual nos permitió conseguir esta estética.

Evidentemente al tratarse de una empresa enfocada en el diseño de trajes, vimos como una prioridad incorporar un elemento grafico representativo a el universo de la costura; la elección fue clara, una aguja de costura, el elemento por excelencia si nos referimos a el mundo de la

moda. En relación con la elección de colores, quisimos utilizar un tono medio del Pantone 19-1617 TCX, concretamente un burdeos, que representa la elegancia y el lujo de una manera silenciosa pero que no deja de ser un color llamativo. Además, consideramos que, a la hora de realizar el proyecto de la marca, y específicamente a la hora de diseñar la página web, pensamos que era un color atractivo, capaz de captar la atención del usuario sin caer en excesos, manteniendo esa misma línea de sobriedad distinguida que buscamos para la identidad visual completa.

Anexo III – Librería externa para gestión de fechas y horas

Para la manipulación y selección de fechas y horas dentro de la interfaz gráfica, se integró la librería externa LGoodDatePicker, una solución robusta y específicamente diseñada para aplicaciones Swing que simplifica notablemente la captura de datos temporales.

III.1. Criterios de selección

La elección de esta librería frente a alternativas nativas como JSpinner, JTextField con validación manual o JDateChooser respondió a varios criterios fundamentales.

En primer lugar, LGoodDatePicker ofrece una integración nativa con las clases del paquete java.time (LocalDate y LocalTime), introducido en Java 8, lo que permite un manejo de fechas más moderno, inmutable y seguro. En segundo lugar, proporciona componentes visuales completos: un DatePicker con calendario desplegable y un TimePicker con menú de horas, ambos completamente personalizables en cuanto a formato, idioma y apariencia.

III.2. Configuración del DatePicker

El DatePicker se implementó en el formulario de creación y edición de citas (Panel_citas) para que el empleado pudiese seleccionar la fecha de forma visual, eliminando los errores asociados a la entrada manual de texto.

```
// Configuración del DatePicker
DatePickerSettings dateSettings = new DatePickerSettings();
dateSettings.setDateRangeLimits(LocalDate.now(), null);
dateSettings.setFormatForDatesCommonEra("yyyy-MM-dd");

DatePicker dpFecha = new DatePicker(dateSettings);
dpFecha.getComponentToggleCalendarButton().setVisible(true);
dpFecha.getComponentToggleCalendarButton().setBackground(new
Color(78, 17, 48));
dpFecha.getComponentToggleCalendarButton().setForeground(Color.WH
ITE);
dpFecha.getComponentDateTextField().setFont(new Font("Tahoma",
Font.PLAIN, 14));
dpFecha.setBounds(475, 100, 205, 32);
```

El formato yyyy-MM-dd se estableció para garantizar la compatibilidad directa con el tipo DATE de MySQL Workbench, evitando conversiones adicionales en la capa Modelo.

Además, se restringió el rango de fechas seleccionables mediante

- `setDateRangeLimits(LocalDate.now(), null)`

lo que impide al usuario escoger fechas pasadas. Esta validación, implementada directamente en la interfaz, refuerza la regla de negocio que exige que las citas solo puedan programarse desde el día actual en adelante, complementando las comprobaciones realizadas en la base de datos.

III.3. Configuración del TimePicker

De la misma manera, el TimePicker se configuró para adaptarse al horario laboral del taller de Edna, que opera de 9:00 a 20:00 horas. Para ello, se empleó la interfaz TimeVetoPolicy, que permite definir qué horas son permitidas y cuáles deben ocultarse o deshabilitarse en el desplegable:

```
// formato de hora minima y maxima
TimePickerSettings ajustes = new TimePickerSettings();

tpHora = new TimePicker(ajustes);
tpHora.getComponentTimeTextField().setFont(new Font("Tahoma", Font.PLAIN, 14));
tpHora.getComponentToggleTimeMenuButton().setBackground(new Color(78, 17, 48));
tpHora.getComponentToggleTimeMenuButton().setForeground(Color.white);
tpHora.setBounds(475, 141, 120, 32);
add(tpHora);

// Implementacion de TimeVetoPolicy para limitar horas mínimas y máximas
ajustes.setVetoPolicy(new TimeVetoPolicy() {
    @Override
    public boolean isTimeAllowed(LocalTime time) {
        // Hora mínima: 09:00, Hora máxima: 20:00
        LocalTime minTime = LocalTime.of(9, 0);
        LocalTime maxTime = LocalTime.of(20, 0);

        // No permitir horas fuera del rango
        return !time.isBefore(minTime) && !time.isAfter(maxTime);
    }
});
```

El formato 24 horas (HH:mm) se adoptó por claridad y para evitar ambigüedades entre AM y PM, manteniendo la coherencia con el almacenamiento en la base de datos, donde el tipo TIME también emplea este formato.

La política de veto horaria implementa directamente la regla de negocio que limita las citas al horario laboral del taller, bloqueando automáticamente cualquier hora fuera del rango establecido.

III.4. Integración con la capa Modelo

Una vez que el empleado confirma el formulario, la capa Controlador recupera los valores seleccionados mediante los métodos:

- `getDateStringOrEmptyString()`
- `getTimeString()`

Estos métodos devuelven objetos `String` que contienen los valores seleccionados en el formato que se les estableció en su declaración en la vista, si no fue seleccionada ninguna fecha u hora este método devuelve un `String` vacío.



```
String fecha = (String) panel_cita.getDpFecha().getDateStringOrEmptyString();  
//este paso a string asegura de que si no hay una hora, el string se llenara con la cadena "n/a"  
String hora = panel_cita.getTpHora().getTimeStringOrSuppliedString("n/a");
```

Estas cadenas se pasan directamente a los métodos `crearCita()` y `modificarCita()` de la clase `ConsultasCita`, que las insertan en la base de datos mediante `PreparedStatement`. De esta forma, la librería externa se integró de manera limpia en la arquitectura MVC, respetando la separación de responsabilidades y evitando que la lógica de formato contaminase otras capas.

III.5. Importación de la librería

La librería LGoodDatePicker no se gestionó mediante un gestor de dependencias como Maven, sino que se incorporó al proyecto como una librería externa en formato .jar. El archivo LGoodDatePicker-11.2.1.jar se descargó desde su repositorio oficial:

- <https://github.com/LGoodDatePicker/LGoodDatePicker>

y se añadió al classpath del proyecto en Eclipse mediante la siguiente ruta: Propiedades del proyecto → Java Build Path → Libraries → Add External JARs.

Este método de importación garantiza que la librería esté siempre disponible en el entorno de desarrollo sin necesidad de conexión a repositorios remotos, y facilita el uso del proyecto al incluir el archivo .jar junto con el código fuente en el repositorio de GitHub.

III.6. Beneficios obtenidos

La inclusión de LGoodDatePicker contribuyó con múltiples ventajas para el desarrollo del proyecto. En primer lugar, la selección gráfica eliminó por completo los errores de formato de fecha y hora que eran frecuentes con la entrada manual de texto. En segundo lugar, la experiencia de usuario mejoró notablemente, ya que los calendarios y menús desplegables resultan más intuitivos y profesionales que los componentes nativos tradicionales.

En tercer lugar, las validaciones de rango y horario se aplican directamente en la capa Vista gracias a las políticas de veto, lo que reduce la carga de la base de datos al evitar comprobaciones innecesarias. Por último, la integración con java.time permitió escribir un código más limpio y menos propenso a errores, eliminando las tediosas conversiones entre Date, Calendar y String que habrían sido necesarias con las alternativas nativas de swing.